

编号: JX/GC7.5-04-JL06

20243301225



学校代码 10857

学 号

分 类 号

密 级 公 开

北京信息职业技术学院

Beijing Information Technology College

毕业设计(论文)

(基于分布式架构的 OpenStack 私有云计算平台的设计与实现)

学历层次	大专
院系名称	产业互联网学院
专业名称	计算机网络
学生姓名	邵宗贤
指导教师	李慧颖

年 月 日

毕业设计(论文)任务书

I、毕业设计(论文)题目

基于分布式架构的 Openstack 私有云计算平台的设计与实现

II、毕业设计(论文)选题意义及要求

平时大家使用人工智能,但是有很多大模型的 Token 还是供不应求,这时候 Openstack 的分布式架构就可以派上用场,及 Token 可快速发放、人工智能模型快速链接来解决单显卡算力不够用的现实,这样物理算力不够用,用多台设备来提升算力。

III、毕业设计(论文)工作内容和进度安排

第一周——第七周 收集数据,获取大模型 Token

第八周 搭建平台

第九周 测试、系统联调

第十周——第十一周 编写论文

第十二周 编写答辩 PPT

IV、主要参考资料

- [1] Jeffery Chukwuma Obiri. Scalable Enterprise Decision-Support Platforms Using Containerized AI Workflows on Kubernetes-Openstack Infrastructure[J]. Global Academic Journal of Economics and Business, 2023, 5(6):172-186.
https://www.researchgate.net/publication/374528987_Scalable_Enterprise_Decision-Support_Platforms_Using_Containerized_AI_Workflows_on_Kubernetes-Openstack_Infrastructure
- [2] 佚名. OpenStack 平台集成指南 [M]. 北京: CSDN 文库, 2026.
<https://wenku.csdn.net/doc/7edvax9p01>
- [3] 佚名. AI 开发新姿势: OneAPI 管理 20+ 主流模型密钥 [J]. CSDN 博客, 2026.
https://blog.csdn.net/weixin_29443363/article/details/158046592
- [4] 佚名. 墨语灵犀部署实录: OpenStack 私有云中高可用大模型集群搭建 [J]. CSDN 博客, 2026. https://blog.csdn.net/weixin_42315569/article/details/157861336

产业互联网 教学系 计算机网络 专业 2433012 班

学生姓名(学号) 邵宗贤 20243301225

毕业设计(论文)时间: 自 2025 年 11 月 日至 2026 年 04 月 日

答辩时间: 2026 年 04 月 18 日 成绩

指导教师: 李慧颖

目录

目录

摘要.....	I
ABSTRACT.....	II
第一章 研究背景与意义.....	1
1.1 研究背景.....	1
1.1.1 研究意义.....	1
1.2 国内外研究现状.....	1
1.2.1 国内研究现状.....	2
1.3 研究内容与方法.....	3
1.3.1 研究内容.....	3
1.3.2 研究方法.....	3
第二章 相关理论与技术基础.....	4
2.1 分布式架构理论.....	4
2.1.1 分布式架构核心概念.....	4
2.1.2 分布式架构的关键特性.....	4
第三章 平台搭建环境准备.....	5
3.1.1 硬件环境配置.....	5
3.1.2 操作系统.....	7
3.1.3 基础依赖软件安装（所有节点执行）.....	7
3.1.4 Python 环境配置（OpenStack 核心运行环境）.....	7
3.1.5 虚拟化与网络工具.....	7
第四章 操作系统与依赖软件安装.....	8
4.1.1 搭建 Openstack.....	8
4.1.2 安装操作系统：.....	8
4.2 基础环境配置.....	30
4.2.1 配置网卡.....	30
4.2.2 计算节点添加硬盘，并进行分区操作.....	31
4.2.3 上传基础镜像.....	33
4.2.4 配置 yum 源文件.....	34
4.2.5 安装 ftp 服务.....	35
4.2.6 安装 iaas-xiandian 服务.....	36
4.2.7 打开 openrc.sh 脚本文件，配置环境变量.....	36
4.3 安装 OpenStack 所需的库和工具.....	37
4.3.1 安装 iaas-pre-host.sh 服务.....	37
4.3.2 安装数据库服务.....	37
4.3.3 安装 Keystone 认证服务.....	37
4.3.4 安装 Glance 镜像服务.....	38
4.3.5 安装 Nova 计算服务.....	38

4.3.6 安装 Neutron 网络服务	38
4.3.7 安装 dashboard 服务	38
4.3.8 安装 Cinder 块存储服务	38
4.3.9 安装 Swift 对象存储服务	38
4.4.1 测试和验证 OpenStack 平台	39
第五章 环境初始化配置	40
5.1.1 关闭防火墙与 SELinux	40
5.1.2 配置主机名与节点解析	40
5.1.3 集群时间同步配置	40
5.1.4 配置静态 IP 与多平面隔离	41
5.1.5 检查硬件虚拟化支持	41
第六章 分布式节点部署与配置	42
6.1 节点角色划分（基于 OpenStack 三节点集群）	42
6.1.1 技术栈选型	42
6.2 OpenStack 分布式节点配置	42
6.2.1 控制节点配置	42
6.2.2 计算节点（GPU 节点）配置	43
6.2.3 加入 OpenStack 集群	43
6.2.4 安装大模型推理依赖	45
6.2.5 OpenStation 可视化部署	46
6.2.6 集成 OpenStack 认证	47
6.2.7 验证与使用	48
第七章 Keystone（身份认证服务）	50
7.1 核心实体	50
7.1.1 Nova（计算服务）	51
7.1.2 Nova 关键配置文件	51
7.2 Neutron（网络服务）	54
7.2.1 Neutron 是干什么的？	54
7.2.2 核心组件	54
7.2.3 核心网络概念	54
7.2.4 关键配置	56
7.2.5 对你大模型部署最重要的 4 件事	57
7.3 Cinder/Swift（存储服务）	58
7.3.1 Cinder（块存储）	58
7.3.2 Swift（对象存储）	58
7.4 Horizon（可视化管理界面）	60
7.4.1 环境前提	60
7.4.2 安装依赖与 Horizon 包	60
7.4.3 核心配置（local_settings）	60
7.4.4 Apache 配置（httpd）	61
结论	63
致谢	64
参考文献	65

摘要

Openstack 随着数字经济快速发展,企业与机构对数据安全、自主可控的 IT 基础设施需求持续提升,私有云成为中小型企业、教育机构、政务部门信息化建设的主流方向。OpenStack 作为开源 IaaS 云计算平台,采用分布式松耦合架构,支持计算、网络、存储资源统一池化管理,具备高可用、易扩展、低成本等优势,是搭建私有云的首选技术方案。本文以分布式架构理论为指导,围绕轻量化、可扩展、易部署的私有云建设目标,设计并实现基于 OpenStack Train 版本的私有云计算平台。研究选用 CentOS Linux 作为底层操作系统,通过 Packstack 部署完成多机环境搭建,并支持平滑扩展为多节点分布式集群。论文系统完成需求分析、架构设计、环境配置、组件部署、功能实现、测试优化全流程,实现用户认证、虚拟机生命周期管理、虚拟网络、块存储、Web 可视化管理等核心功能。经测试,平台功能完整、运行稳定,虚拟机创建成功率 100%,并发调度响应迅速,满足中小型场景私有云使用需求。研究成果形成标准化部署方案,可为同类单位提供可直接复用的私有云建设参考。

关键词: 分布式架构; OpenStack; 私有云; 云计算; 虚拟化; CentOS

ABSTRACT

Openstack the rapid development of digital economy, enterprises and institutions have an increasing demand for data security and self-controlled IT infrastructure. Private cloud has become the mainstream direction of information construction for small and medium-sized enterprises, educational institutions and government departments. As an open-source IaaS cloud computing platform, OpenStack adopts a distributed loosely coupled architecture, supports unified pooling management of computing, network and storage resources, and has the advantages of high availability, easy expansion and low cost. It is the preferred technical solution for building private cloud.

Guided by the theory of distributed architecture, this paper designs and implements a private cloud computing platform based on OpenStack Train version around the private cloud construction goals of lightweight, scalable and easy to deploy. The research selects CentOS Linux as the underlying operating system, completes the stand-alone All-in-One environment deployment through Packstack one-click deployment tool, and supports smooth expansion to multi-node distributed cluster. The paper systematically completes the whole process of demand analysis, architecture design, environment configuration, component deployment, function implementation and test optimization, and realizes the core functions such as user authentication, virtual machine lifecycle management, virtual network, block storage and Web visual management.

After testing, the platform has complete functions and stable operation. The success rate of virtual machine creation is 100%, and the concurrent scheduling response is rapid, which meets the needs of private cloud in small and medium-sized scenarios. The research results form a standardized deployment scheme, which can provide a directly reusable private cloud construction reference for similar institutions.

Key words: Distributed Architecture; OpenStack; Private Cloud; Cloud Computing; Virtualization; CentOS

第一章 研究背景与意义

1.1 研究背景

随着云计算、大数据、人工智能等新一代信息技术的快速发展，传统 IT 架构正经历着深刻变革。从早期的物理服务器托管，到虚拟化集中管理，再到云端弹性调度，信息化建设逐步向资源池化、服务化、自动化方向演进。IaaS（基础设施即服务）层作为云计算的底层核心，负责提供计算、存储、网络等基础资源，是上层业务运行的基础支撑。在数据安全、合规监管、成本控制等多重因素驱动下，越来越多的机构放弃纯公有云方案，转向私有云或混合云架构。私有云具备数据本地存储、网络自主管控、权限独立配置、长期成本更低等优势，更适合对数据敏感、安全性要求高的场景。开源技术的成熟极大地降低了私有云建设门槛，OpenStack 凭借其社区活跃、组件齐全、生态完善、支持分布式扩展等特点，成为行业事实标准。

1.1.1 研究意义

云计算、大数据、人工智能等新一代信息技术正在深刻改变传统 IT 架构。从早期物理服务器托管，到虚拟化集中管理，再到云端弹性调度，信息化建设逐步向资源池化、服务化、自动化方向演进。IaaS 层作为云计算底层核心，负责提供计算、存储、网络等基础资源，是上层业务运行的基础支撑。

在数据安全、合规监管、成本控制等多重因素驱动下，越来越多机构放弃纯公有云方案，转向私有云或混合云架构。私有云具备数据本地存储、网络自主管控、权限独立配置、长期成本更低等优势，更适合对数据敏感、安全性要求高的场景。

开源技术的成熟极大地降低了私有云建设门槛。OpenStack、CloudStack、OpenNebula 等平台中，OpenStack 凭借社区活跃、组件齐全、生态完善、支持分布式扩展等特点，成为行业事实标准。它能够从单机实验环境平滑扩展到大规模生产集群，适配教学、研发、生产等全场景需求。

1.2 国内外研究现状

国外在云计算及 OpenStack 领域的研究起步较早，技术成熟且应用广泛。许多大型企业和机构，如 Rackspace、NASA、IBM、HP 等，都是 OpenStack 的早期贡献者和使用者。这些企业和机构通过 OpenStack 构建了大规模的私有云和混合云环境，积累了丰富的实践经验。同时，国外在分布式架构、虚拟化技术、网络虚拟化、存储技术等方面的研究也为 OpenStack 的发展提供了有力支持。

1.2.1 国内研究现状

近年来，随着国内信息化建设的不断推进，云计算技术得到了广泛应用。国内许多企业和机构，如华为、腾讯、阿里、中国移动等，都在积极研究和应用 OpenStack 技术，构建私有云和混合云环境。同时，国内高校和科研机构也在积极开展云计算和 OpenStack 相关的研究工作，取得了一系列成果。然而，与国外相比，国内在 OpenStack 的深度应用和技术创新方面仍存在一定差距，需要进一步加强研究和探索。国内云计算产业快速发展，华为、腾讯、阿里、中国移动等企业积极投入 OpenStack 研发与落地，推出面向政企的私有云和混合云产品。高校与科研机构围绕分布式调度、性能优化、安全加固等方向开展研究，取得一系列成果。

1.3 研究内容与方法

1.3.1 研究内容

1. 研究分布式架构、云计算、虚拟化与 OpenStack 核心技术。
 2. 完成私有云平台需求分析，设计分布式物理架构与逻辑架构。
 3. 基于 CentOS 7 与 OpenStack Train 完成控制节点、计算节点、存储节点部署。
 4. 实现 Keystone、Nova、Neutron、Cinder、Swift、Horizon 等核心组件配置与联调。
 5. 开展功能测试、性能测试、稳定性测试，对平台进行优化。
- 总结方案，形成标准化部署文档与可复用成果

1.3.2 研究方法

1. 文献梳理法：梳理云计算、分布式架构、OpenStack 相关标准、分析 Openstack 技术文档。
2. 实验法：使用 VMware Workstation Pro 来搭建实验环境，完成 CentOS Linux 系统安装、配置、调试与测试
3. 架构设计法：遵循分布式松耦合原则，完成分层架构与节点规划。
4. 测试分析法：设计测试 Openstack 的稳定性，量化评估平台功能、性能与稳定性。

第二章 相关理论与技术基础

本章将系统阐述支撑本论文研究工作的核心理论与关键技术。首先介绍分布式架构的基本原理，它是整个平台设计的指导思想；其次分析云计算与私有云的基础概念；然后重点剖析 OpenStack 的核心架构与组件；最后简要说明搭建过程中涉及的虚拟化、网络与存储等配套技术。

2.1 分布式架构理论

分布式架构是一种将系统组件部署在不同网络计算机上，并通过消息传递进行通信和协调的软件设计模式。它旨在通过资源共享和协同工作，实现更高的性能、可靠性和可扩展性。

2.1.1 分布式架构核心概念

分布式系统的核心在于“分布”与“协同”。其核心概念包括：

1. 节点与网络：系统中的每个独立计算单元称为节点，它们通过网络互联。节点可以是物理服务器或虚拟机。
2. 松耦合：各组件之间依赖关系较弱，通过定义良好的接口（如 API）进行通信，从而允许组件独立开发、部署和扩展。
3. 可扩展性：系统能够通过增加资源（如服务器节点）来应对增长的工作负载，包括垂直扩展（增强单节点能力）和水平扩展（增加节点数量）。
4. 容错性与高可用：系统设计允许单个或多个组件发生故障而不导致整体服务中断，通常通过冗余、复制和故障转移机制实现。
5. 一致性、可用性与分区容忍性（CAP 定理）：在分布式系统中，无法同时完全保证数据一致性系统可用性和网络分区容忍性，设计时需要根据应用场景进行权衡。

2.1.2 分布式架构的关键特性

分布式架构的关键特性主要包括容错性、可扩展性、透明性、数据一致性与高性能。这些特性共同构成了分布式系统应对复杂业务挑战、提供可靠服务的基础。以下是这些关键特性的详细说明：

特性	核心含义与重要性
1. 容错性/ 高可用性	系统部分节点失效时，整体服务仍能正常运行。这是通过冗余（多副本）、故障检测与自动恢复等机制实现的，是保障业务连续性的基石。
2. 可扩展	系统能通过增加资源（如服务器）来平滑提升处理能力，以应对增

特性	核心含义与重要性
性	长的业务负载。包括 纵向扩展 （提升单机能力）和更核心的 横向扩展 （增加机器数量）。
3. 透明性	对用户和应用程序隐藏分布式系统的复杂性，使其像使用单一计算机系统一样。主要包括访问透明、位置透明、并发透明等。
4. 数据一致性	在数据多副本存储的场景下，保障所有用户读取到的数据都是符合预期的。需要在一致性、可用性和分区容错性之间进行权衡（如 CAP 定理）。
5. 高性能	通过将任务并行分散到多个节点上处理，以及使服务就近部署（如 CDN），来降低延迟、提高吞吐量，实现单机系统无法达到的性能水平。

第三章 平台搭建环境准备

3.1.1 硬件环境配置

Openstack私有云计算平台分布式架构设计

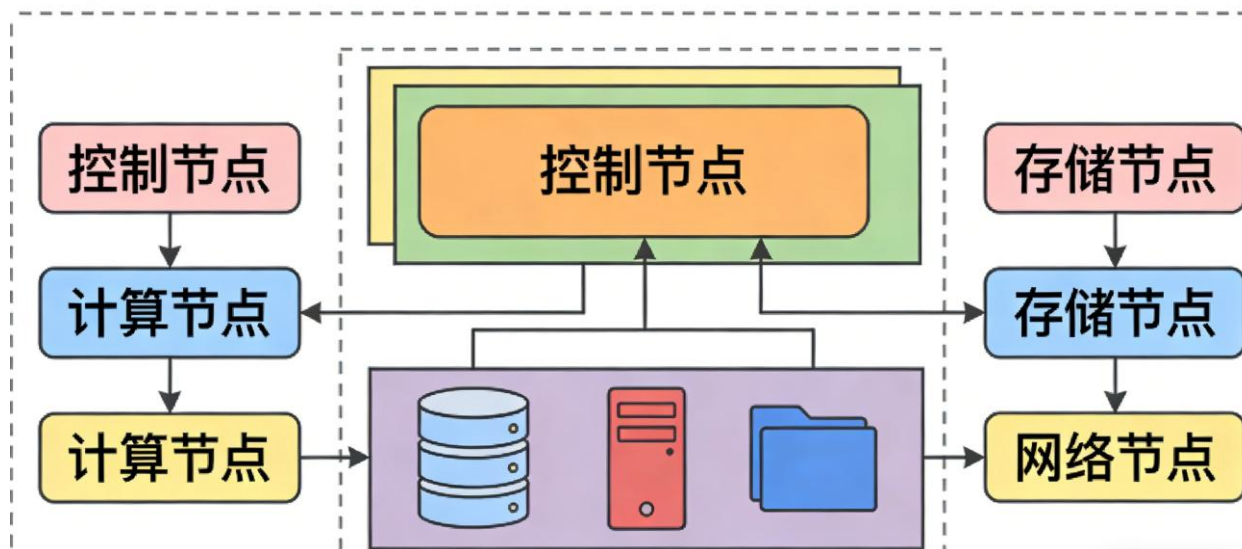


图 3-1 系统模块图

- 控制节点（3 台，奇数，保证集群高可用）
 - CPU: ≥ 16 核，支持 Intel VT-x/AMD-V 虚拟化加速
 - 内存: $\geq 64\text{GB}$
 - 硬盘: SSD 500GB（系统 + 日志 + 数据库）

- 网卡：双万兆以太网卡（支持链路聚合）
- 用途：API 服务、认证、调度、数据库、消息队列、负载均衡
- **计算节点（2 台及以上，可水平扩展）**
 - CPU：≥32 核，开启硬件虚拟化
 - 内存：≥128GB
 - 硬盘：SSD 1TB + HDD 4TB
 - 网卡：双万兆以太网卡
 - 用途：运行虚拟机、提供计算资源
- **存储节点（3 台，Ceph 分布式存储）**
 - CPU：≥8 核
 - 内存：≥32GB
 - 硬盘：SSD 256GB（系统）+ HDD 4TB×4（数据盘）
 - 网卡：万兆以太网卡
 - 用途：提供块存储、镜像存储、数据持久化
- **网络设备**
 - 万兆交换机 1 台
 - 支持 VLAN、端口聚合、网络平面隔离
 - 划分管理网、数据网、存储网、外部业务网

3.1.2 操作系统

所有节点统一安装 CentOS 7 / CentOS Stream 8 / Ubuntu 20.04 LTS 64 位服务器版，最小化安装，无图形界面，保证系统稳定、安全、轻量。

3.1.3 基础依赖软件安装（所有节点执行）

```
# CentOS 系统
yum install -y net-tools vim wget ntp chrony qemu-kvm libvirt libvirt-python \
python3 python3-pip openvswitch openstack-selinux

# Ubuntu 系统
apt install -y net-tools vim wget chrony qemu-kvm libvirt-daemon-system \
libvirt-clients python3 python3-pip openvswitch-switch
```

3.1.4 Python 环境配置（OpenStack 核心运行环境）

```
# 查看 Python 版本
python3 -V

# 安装 pip 并升级
pip3 install --upgrade pip

# 安装 OpenStack 命令行客户端
pip install python-openstackclient python-neutronclient python-novaclient

# 设置 Python3 为默认版本
ln -s /usr/bin/python3 /usr/bin/python
ln -s /usr/bin/pip3 /usr/bin/pip
```

3.1.5 虚拟化与网络工具

```
# 启动并开机自启 libvirt
systemctl enable --now libvirtd

# 启动并开机自启 openvswitch
systemctl enable --now openvswitch
```

第四章 操作系统与依赖软件安装

4.1.1 搭建 Openstack

1. 准备硬件:

确保硬件配置满足基本要求, 例如至少 4GB 内存、足够的处理器和硬盘空间。

2. 准备软件:

软件 1. CentOS-7-x86_64-DVD-1804.iso

软件 2. XianDian-IaaS-v2.2.iso

软件 3: VMware-Workstation-14.1.0.exe

软件 4: SecureCRT&FXPortable7.0.0.326.7z

4.1.2 安装操作系统:

(1) 安装前 Vm 配置:

首先, 在**网络-属性-更改适配器设置**界面中找到两张网卡: **VM-net1** 和 **VM-net8**

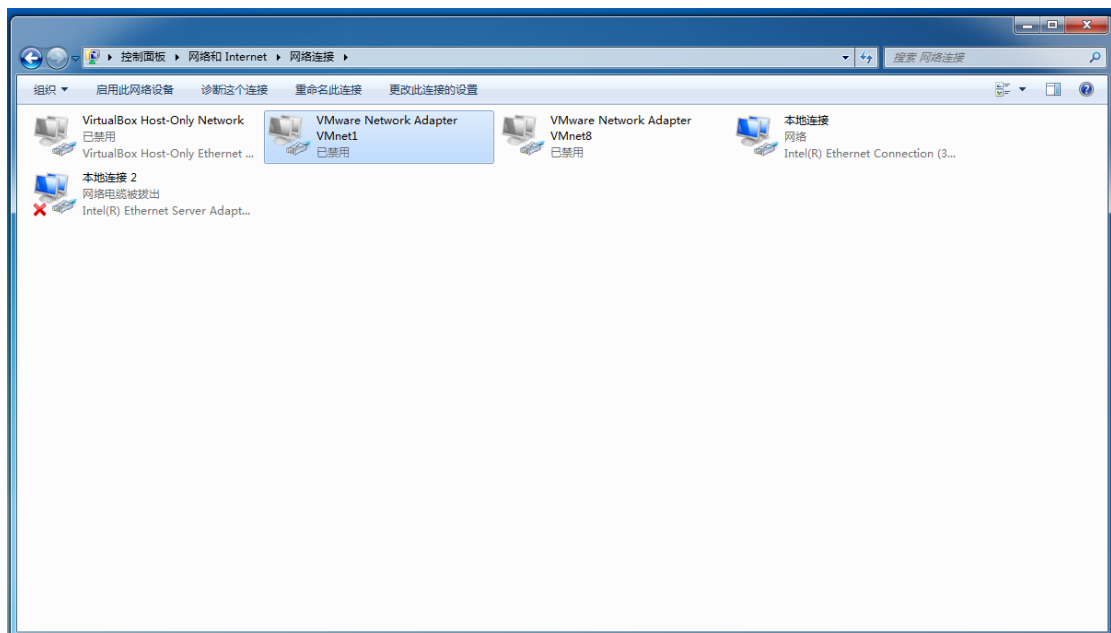
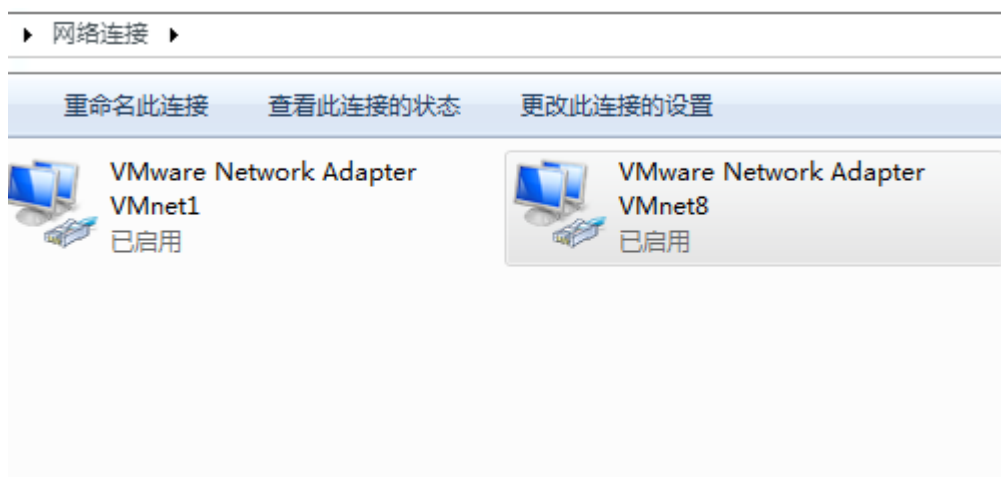
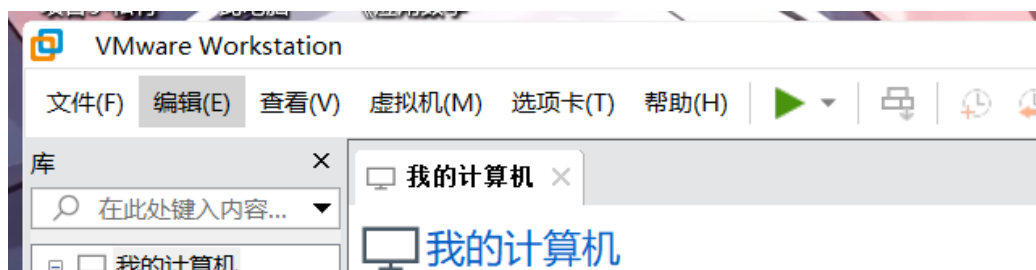


图 3-2 虚拟网卡配置

右键点击这两张网卡并且**启用**这两张网卡:



在编辑菜单下，选择虚拟网络编辑器，将原有的都删除，并添加网络配置如下：



名称 类型 外部连接 主机连接 DHCP 子网地址

VMnet1	仅主机...	-	已连接	-	192.168.100.0
VMnet8	NAT 模式	NAT 模式	已连接	-	192.168.200.0

添加网络(E)... 移除网络(O) 重命名网络(W)...

VMnet 信息

☐ 桥接模式(将虚拟机直接连接到外部网络)(B)

已桥接至(G): 自动设置(U)...

☐ NAT 模式(与虚拟机共享主机的 IP 地址)(N) NAT 设置(S)...

☒ 仅主机模式(在专用网络内连接虚拟机)(H)

☒ 将主机虚拟适配器连接到此网络(V)

主机虚拟适配器名称: VMware 网络适配器 VMnet1

☐ 使用本地 DHCP 服务将 IP 地址分配给虚拟机(D) DHCP 设置(P)...

子网 IP (I): 子网掩码(M):

⚠ 需要具备管理员特权才能修改网络配置。 更改设置(C)

还原默认设置(R) 导入(T)... 导出(X)... 确定 取消 应用(A) 帮助

虚拟网络编辑器

名称	类型	外部连接	主机连接	DHCP	子网地址
VMnet1	仅主机...	-	已连接	-	192.168.100.0
VMnet8	NAT 模式	NAT 模式	已连接	-	192.168.200.0

添加网络(E)... 移除网络(O) 重命名网络(W)...

VMnet 信息

☐ 桥接模式(将虚拟机直接连接到外部网络)(B)
 已桥接至(G): 自动设置(U)...

☒ NAT 模式(与虚拟机共享主机的 IP 地址)(N) NAT 设置(S)...

☐ 仅主机模式(在专用网络内连接虚拟机)(H)

☒ 将主机虚拟适配器连接到此网络(V)
 主机虚拟适配器名称: VMware 网络适配器 VMnet8

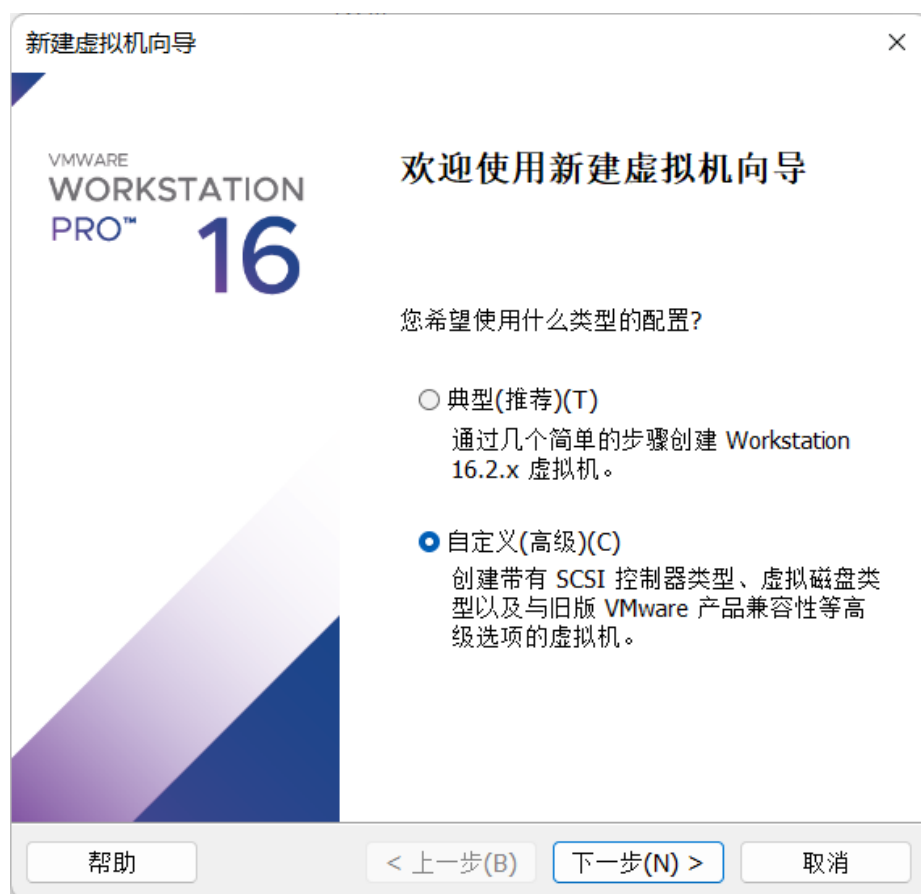
☐ 使用本地 DHCP 服务将 IP 地址分配给虚拟机(D) DHCP 设置(P)...

子网 IP (I): 子网掩码(M):

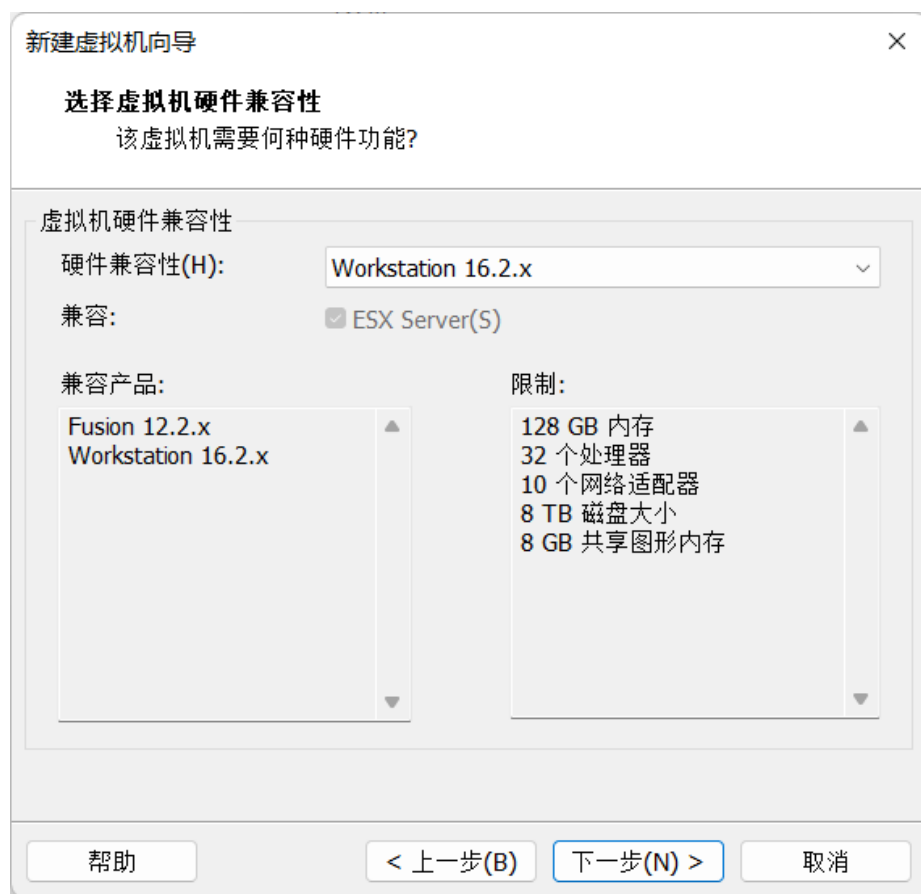
⚠ 需要具备管理员特权才能修改网络配置。 更改设置(C)

还原默认设置(R) 导入(T)... 导出(X)... 确定 取消 应用(A) 帮助

新建虚拟机



下一步



下一步

新建虚拟机向导

安装客户机操作系统

虚拟机如同物理机，需要操作系统。您将如何安装客户机操作系统？

安装来源:

☐ 安装程序光盘(D):

无可用驱动器

☐ 安装程序光盘映像文件(iso)(M):

C:\CentOS-7-x86_64-DVD-151

浏览(R)...

☒ 稍后安装操作系统(S)。

创建的虚拟机将包含一个空白硬盘。

帮助

< 上一步(B)

下一步(N) >

取消

新建虚拟机向导

选择客户机操作系统

此虚拟机中将安装哪种操作系统？

客户机操作系统

☐ Microsoft Windows(W)

☒ Linux(L)

☐ VMware ESX(X)

☐ 其他(O)

版本(V)

CentOS 7 64 位

帮助

< 上一步(B)

下一步(N) >

取消

下一步

编辑虚拟机名称为 controller

新建虚拟机向导



命名虚拟机

您希望该虚拟机使用什么名称?

虚拟机名称(V):

位置(L):

在“编辑”>“首选项”中可更改默认位置。

< 上一步(B)

下一步(N) >

新建虚拟机向导



处理器配置

为此虚拟机指定处理器数量。

处理器

处理器数量(P):

每个处理器的内核数量(C):

处理器内核总数:

1

< 上一步(B)

下一步(N) >

新建虚拟机向导

此虚拟机的内存
您要为此虚拟机使用多少内存?

指定分配给此虚拟机的内存量。内存大小必须为 4 MB 的倍数。

128 GB
64 GB
32 GB
16 GB
8 GB
4 GB
2 GB
1 GB
512 MB
256 MB
128 MB
64 MB
32 MB
16 MB
8 MB
4 MB

此虚拟机的内存(M):
8192 MB

最大推荐内存:
13.4 GB

推荐内存:
1 GB

客户机操作系统最低推荐内存:
512 MB

帮助

< 上一步(B)

下一步(N) >

取消

新建虚拟机向导

网络类型
要添加哪类网络?

网络连接

☐ 使用桥接网络(R)
为客户机操作系统提供直接访问外部以太网网络的权限。客户机在外部网络上必须有自己的 IP 地址。

☒ 使用网络地址转换(NAT)(E)
为客户机操作系统提供使用主机 IP 地址访问主机拨号连接或外部以太网网络连接的权限。

☐ 使用仅主机模式网络(H)
将客户机操作系统连接到主机上的专用虚拟网络。

☐ 不使用网络连接(T)

帮助

< 上一步(B)

下一步(N) >

取消

15

新建虚拟机向导

×

选择 I/O 控制器类型

您希望将哪种 SCSI 控制器类型用于 SCSI 虚拟磁盘?

I/O 控制器类型

SCSI 控制器:

☐ BusLogic(U) (不适用于 64 位客户机)

☒ LSI Logic(L) (推荐)

☐ LSI Logic SAS(S)

☐ 准虚拟化 SCSI(P)

帮助

< 上一步(B)

下一步(N) >

取消

新建虚拟机向导

×

选择磁盘

您要使用哪个磁盘?

磁盘

☒ 创建新虚拟磁盘(V)

虚拟磁盘由主机文件系统上的一个或多个文件组成，客户机操作系统会将其视为单个硬盘。虚拟磁盘可在一台主机上或多台主机之间轻松复制或移动。

☐ 使用现有虚拟磁盘(E)

选择此选项可重新使用以前配置的磁盘。

☐ 使用物理磁盘 (适用于高级用户)(P)

选择此选项可为虚拟机提供直接访问本地硬盘的权限。需要具有管理员特权。

帮助

< 上一步(B)

下一步(N) >

取消

新建虚拟机向导

指定磁盘容量

磁盘大小为多少?

最大磁盘大小 (GB)(S):

128

针对 CentOS 7 64 位 的建议大小: 20 GB

☐ 立即分配所有磁盘空间(A)。

分配所有容量可以提高性能，但要求所有物理磁盘空间立即可用。如果不立即分配所有空间，虚拟磁盘的空间最初很小，会随着您向其中添加数据而不断变大。

☒ 将虚拟磁盘存储为单个文件(O)

☐ 将虚拟磁盘拆分成多个文件(M)

拆分磁盘后，可以更轻松地在计算机之间移动虚拟机，但可能会降低大容量磁盘的性能。

帮助

< 上一步(B)

下一步(N) >

取消

新建虚拟机向导

指定磁盘文件

您要在何处存储磁盘文件?

磁盘文件(F)

将使用此文件名创建一个 128 GB 磁盘文件。

controller.vmdk

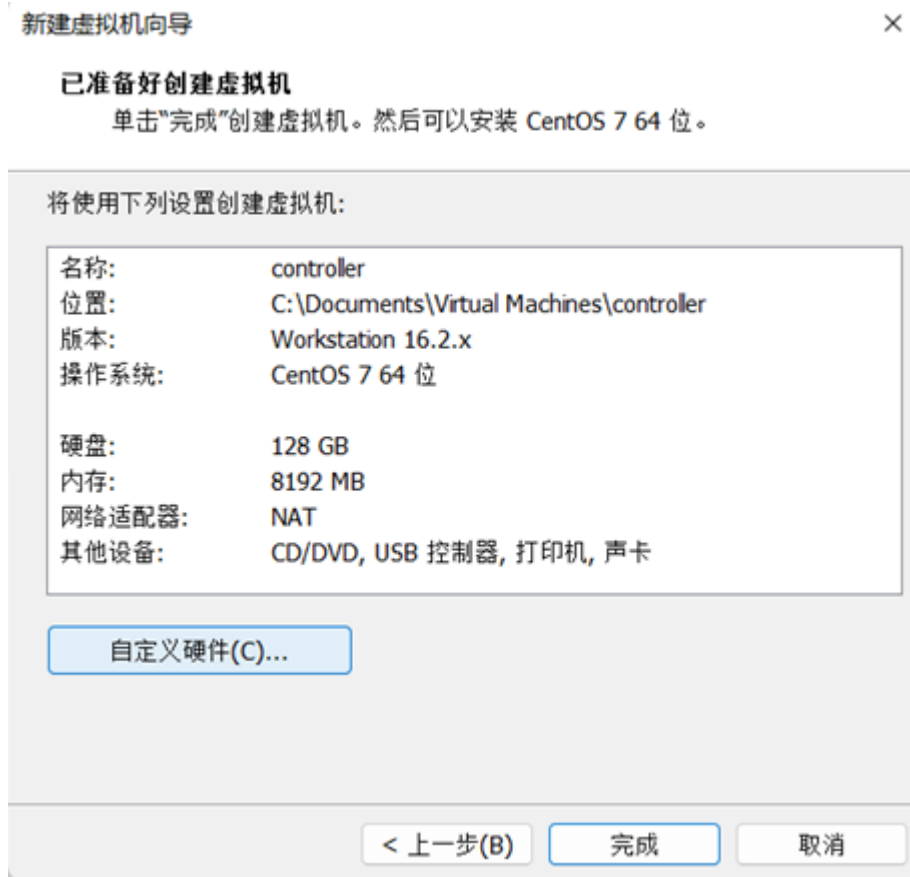
浏览(R)...

帮助

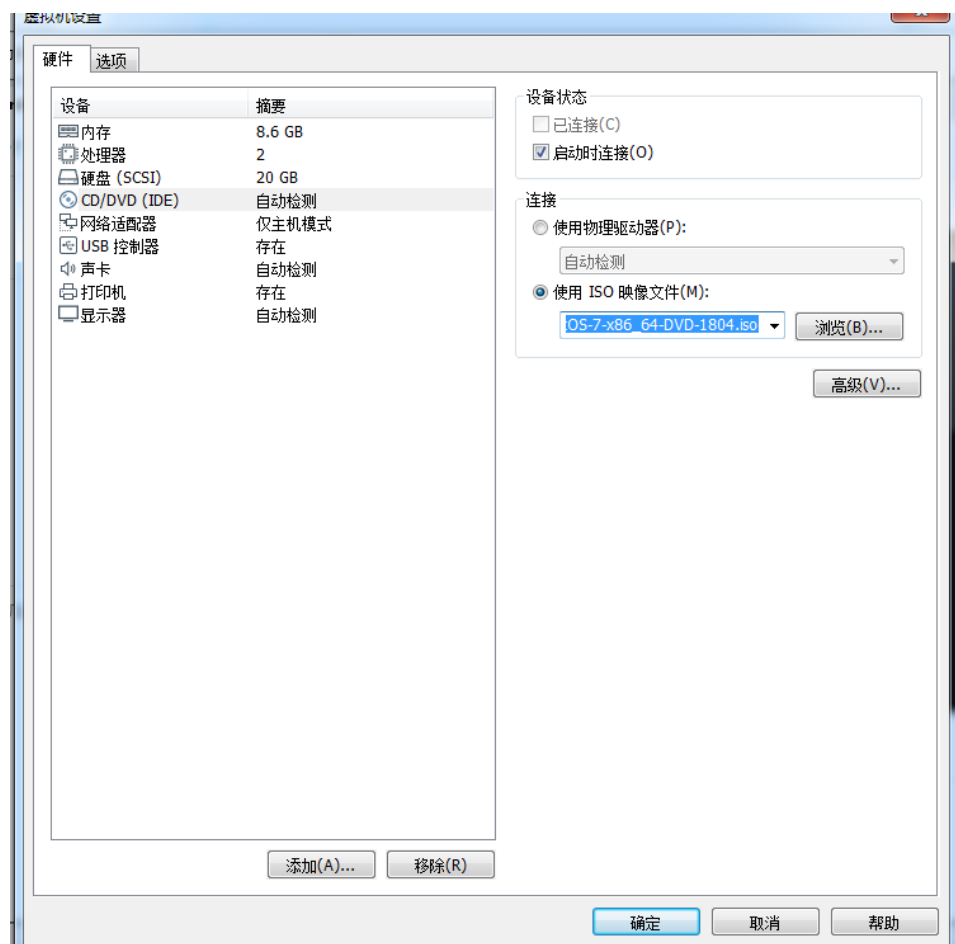
< 上一步(B)

下一步(N) >

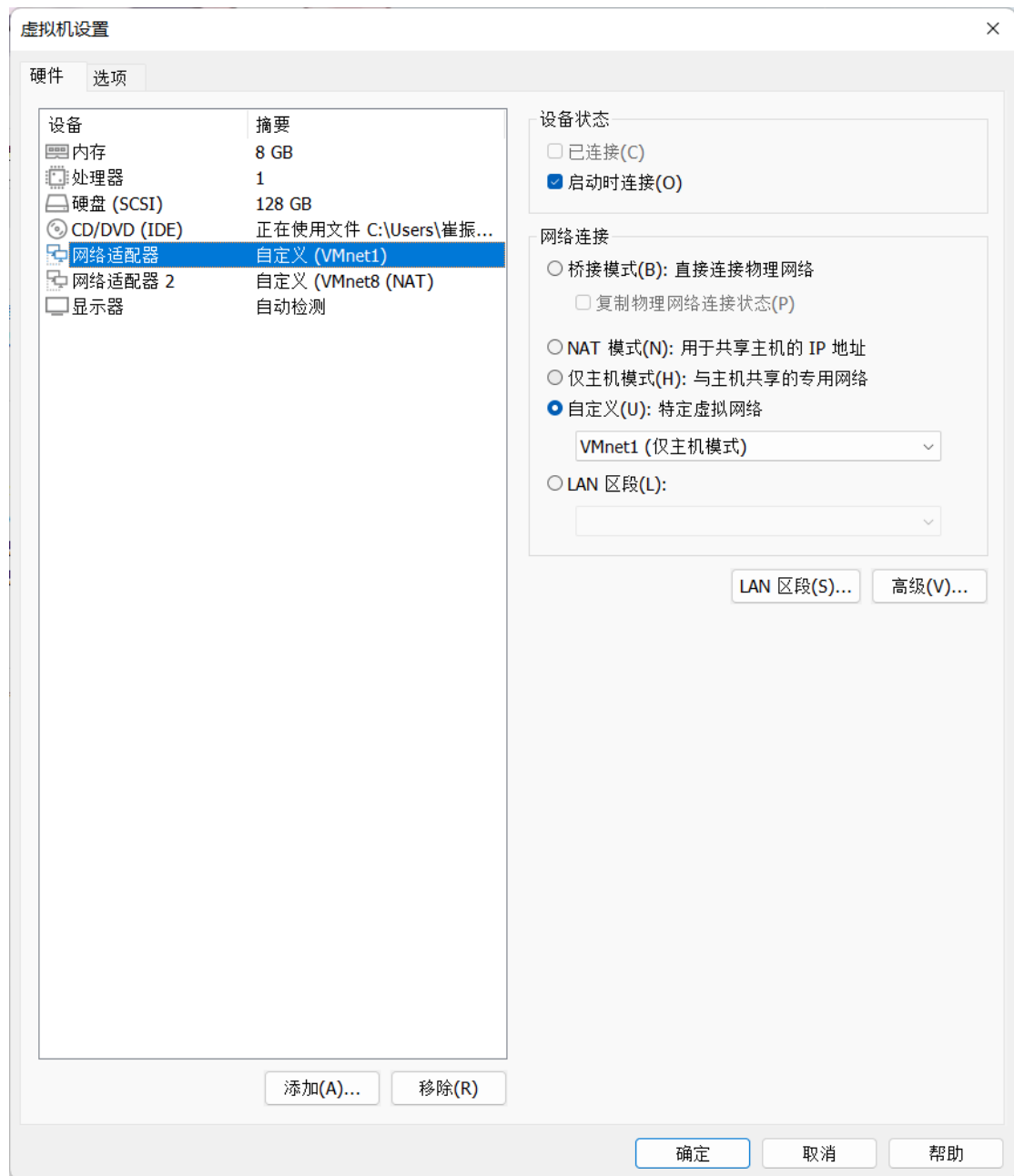
取消



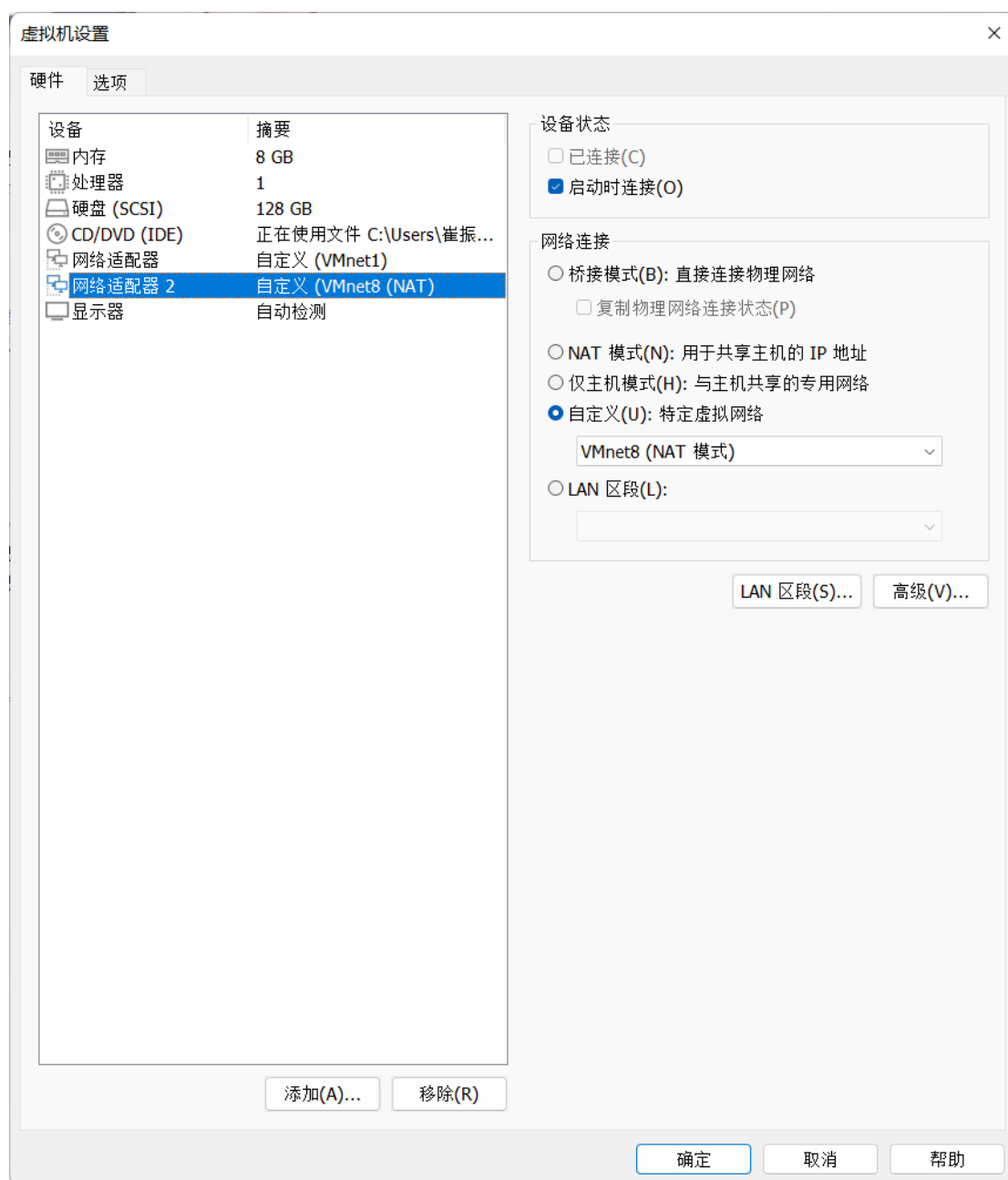
点击自定义硬件，将 usb 控制器，声卡，打印机移除
点击 CD 配置 ios 镜像(1804 版本)



添加网络适配器，并设置第一张网卡为 vmnet1（仅主机模式）



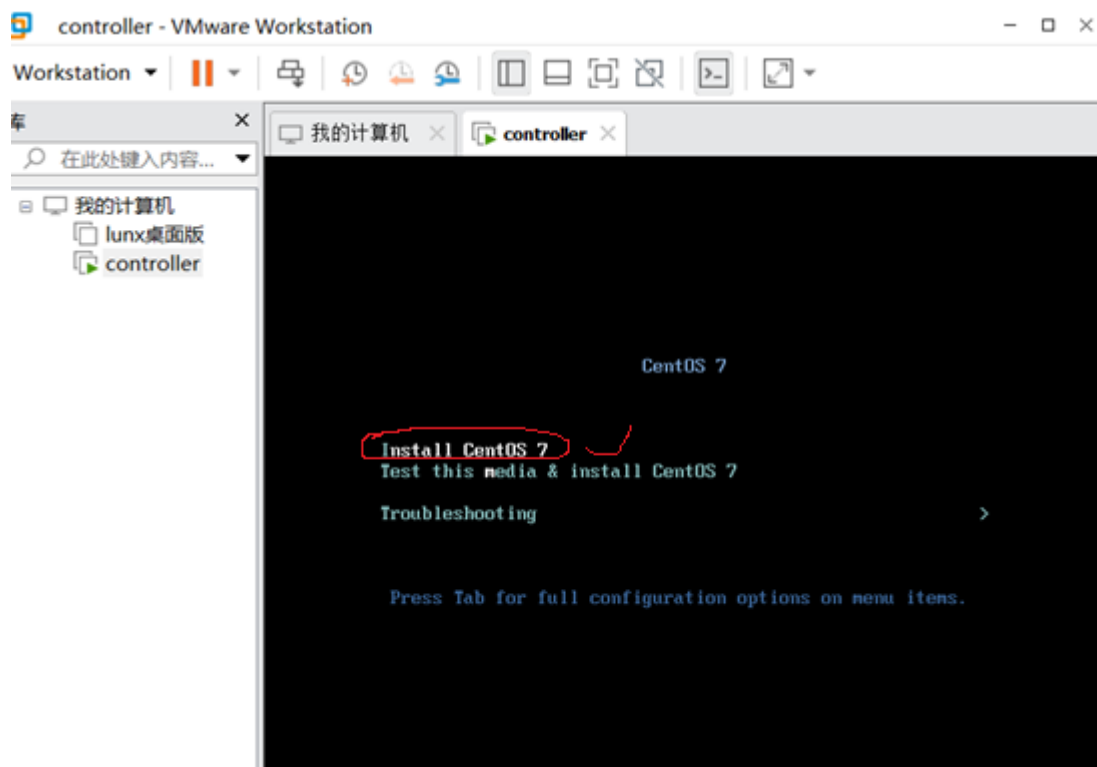
第二张网卡为 VMnet8(net 模式)



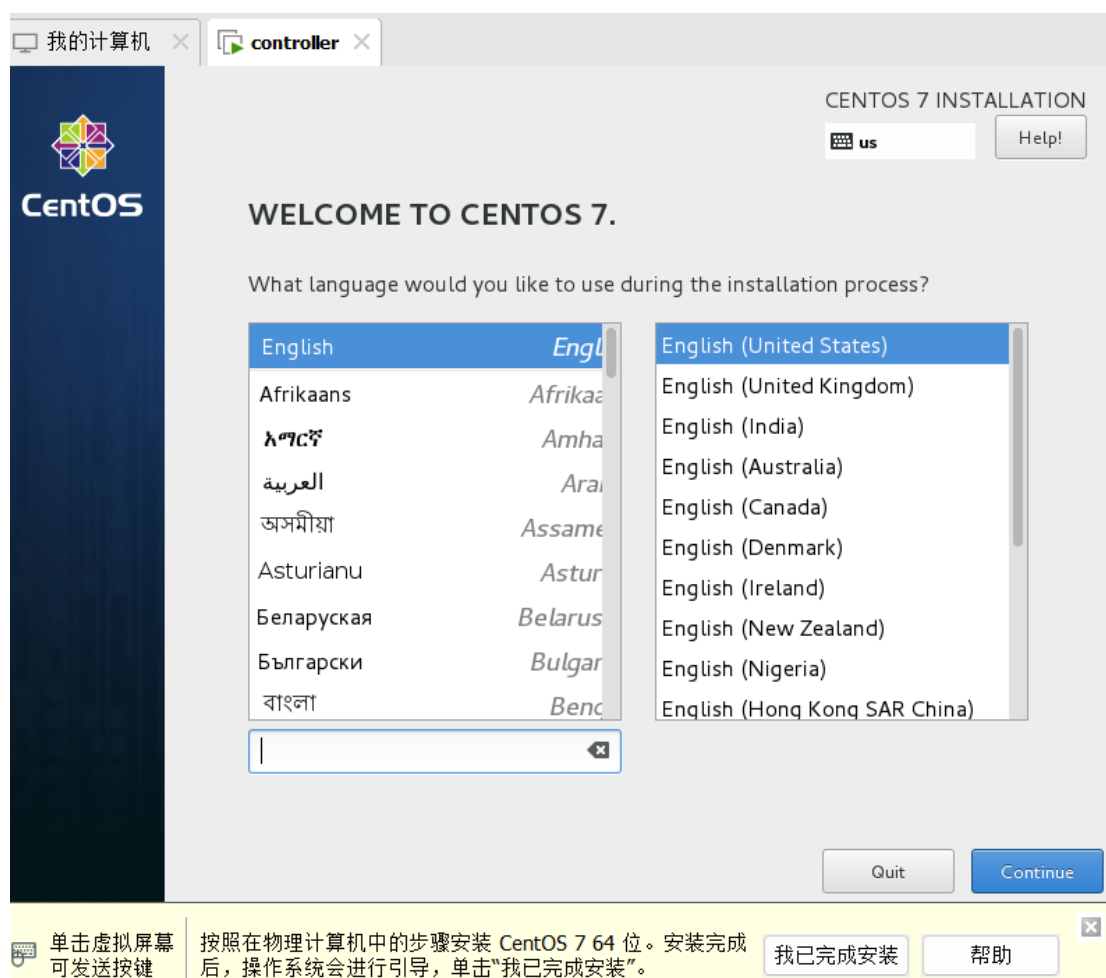
点击确定 再点击完成



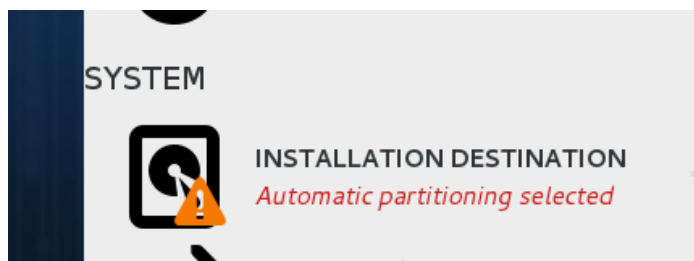
(2) 开始安装操作系统



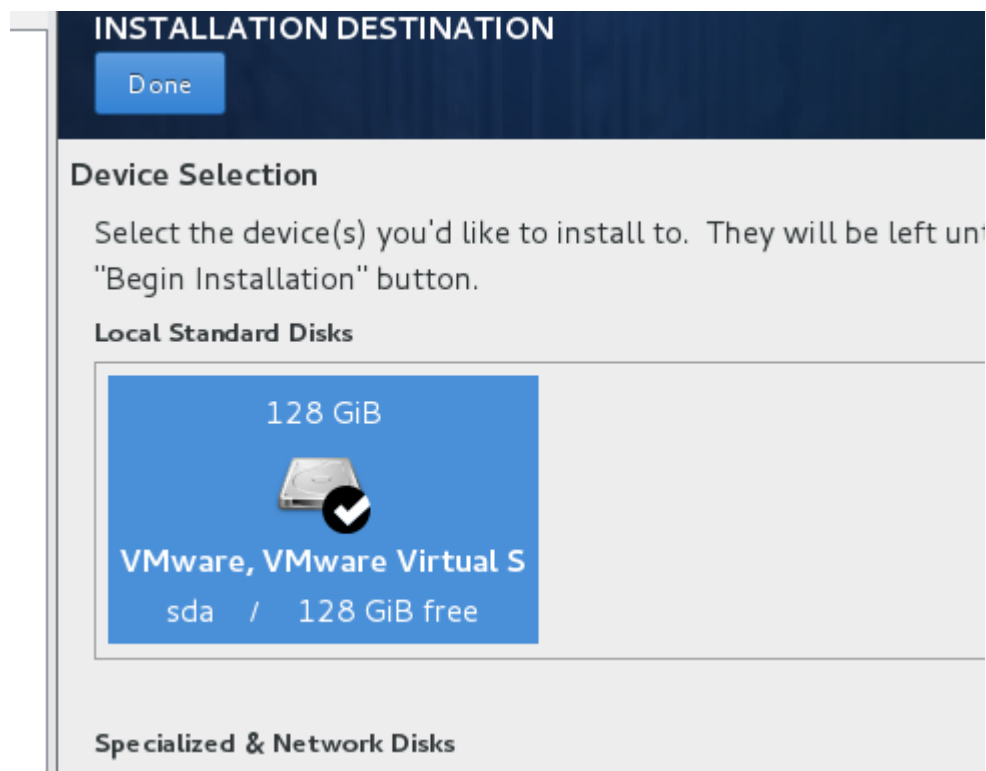
选择语言：英语模式（english）



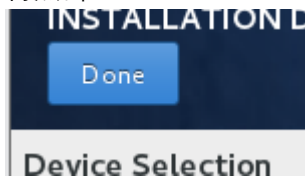
点击 Continue，进入分区模式：

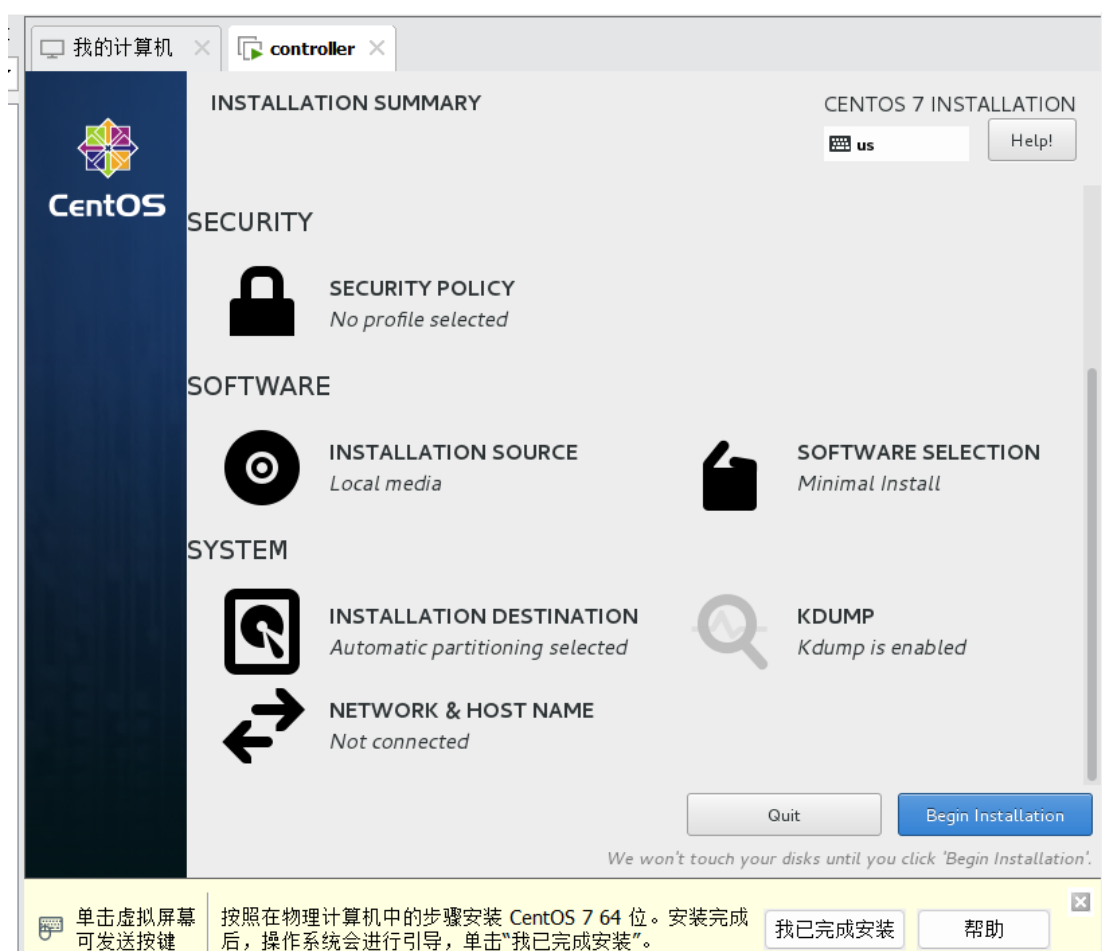


再点击这个盘

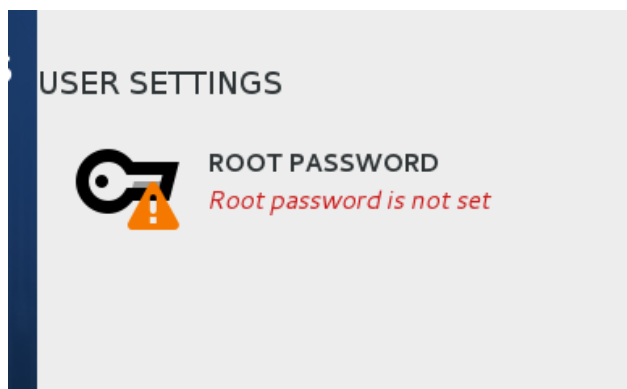


再点击 Done

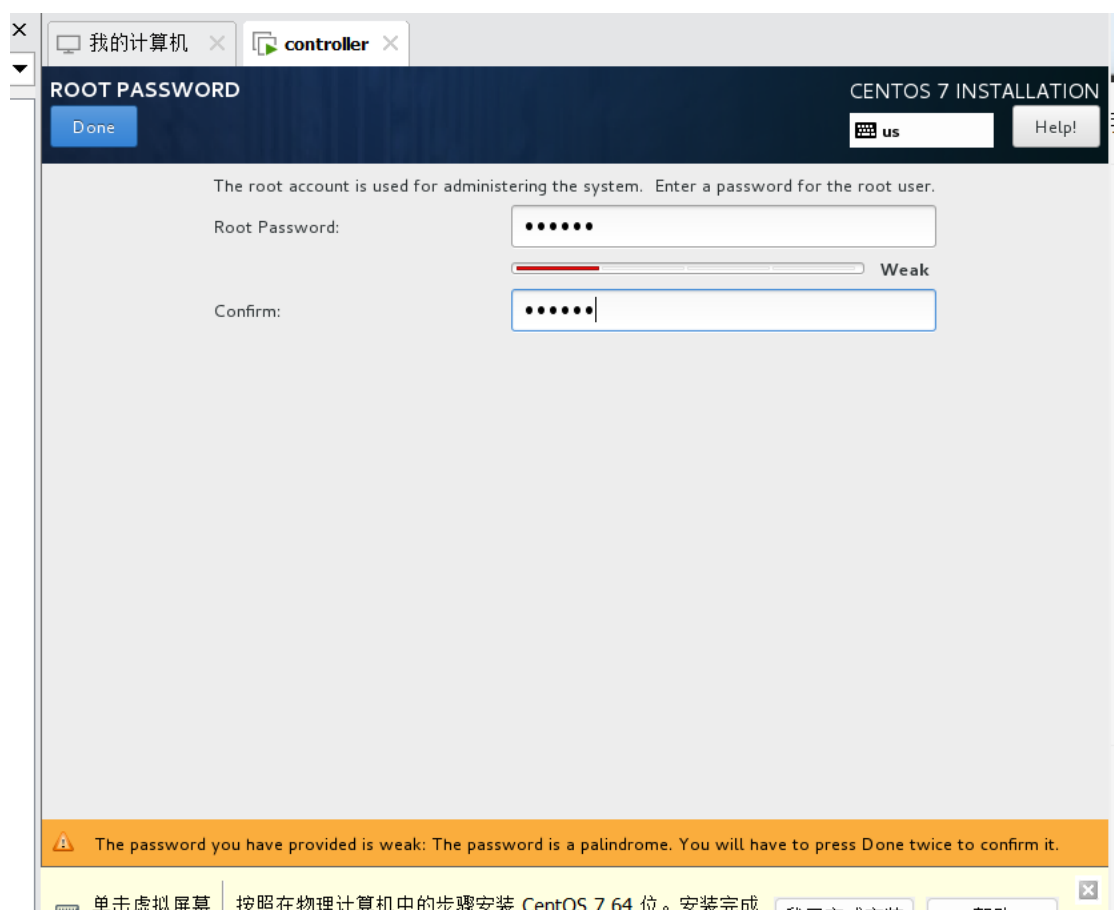




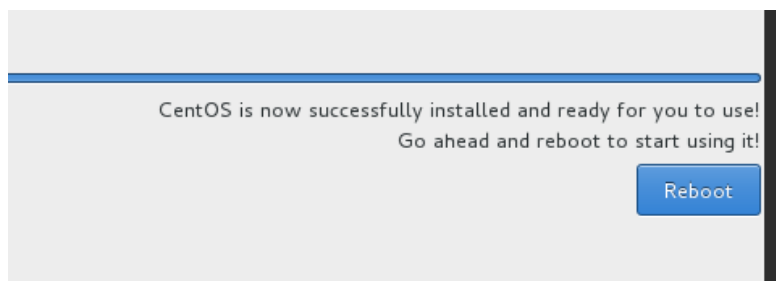
点击 Begin installation, 随后进入 root 用户的密码设置:



输入密码为六个 0 (方便记忆)



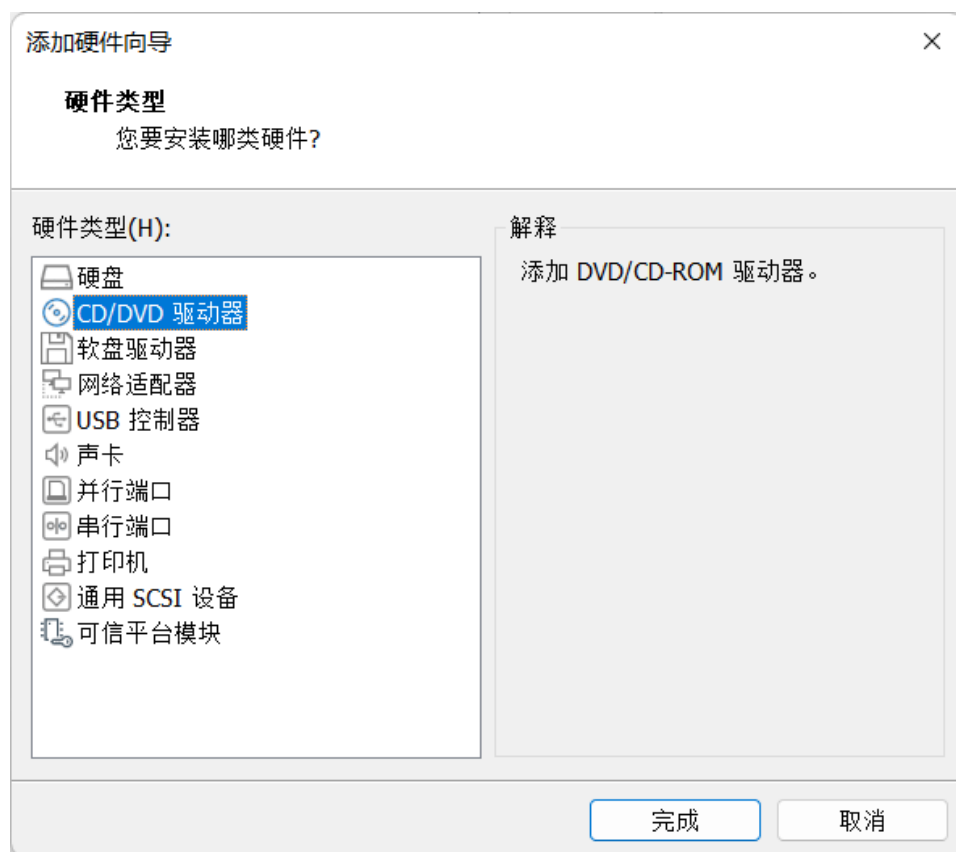
单击虚拟屏幕，按照在物理计算机中的步骤安装 CentOS 7 64 位。安装完成后，等待虚拟机安装，安装完成后右下角会出现 Reboot，这时表明已安装成功。



我们点击 Reboot，重启系统：



系统正常进入后，关闭此已经安装好的虚拟机，进入虚拟机设置模式。编辑虚拟机设置添加 cd，连接已经准备好的软件 iaas 的 ISO 文件。





此阶段 controller（控制节点）虚拟机已安装完毕。

(3) Compute 计算节点操作系统的安装

Compute 计算节点操作系统的安装和 Controller 控制节点操作系统的安装大致相同，不过如下两处有更改

新建虚拟机向导

处理器配置

为此虚拟机指定处理器数量。

处理器

处理器数量(P): 2

每个处理器的内核数量(C): 2

处理器内核总数: 4

帮助

< 上一步(B)

下一步(N) >

取消

新建虚拟机向导

此虚拟机的内存

您要为此虚拟机使用多少内存?

指定分配给此虚拟机的内存量。内存大小必须为 4 MB 的倍数。

128 GB -

64 GB -

32 GB -

16 GB -

8 GB -

4 GB -

2 GB -

1 GB -

512 MB -

256 MB -

128 MB -

64 MB -

32 MB -

16 MB -

8 MB -

4 MB -

此虚拟机的内存(M):

4096 MB

最大推荐内存:
13.4 GB

推荐内存:
1 GB

客户机操作系统最低推荐内存:
512 MB

帮助

< 上一步(B)

下一步(N) >

取消

4.2 基础环境配置

4.2.1 配置网卡

打开 Controller 节点虚拟机,利用 VI 编辑器打开网卡文件,此网卡文件对应的 VMNET1 网卡,并按照下图编辑并保存。

```
vi /etc/sysconfig/network-scripts/ifcfg-ens33
```

```
TYPE=Ethernet
BOOTPROTO=static
DEFROUTE=yes
PEERDNS=yes
PEERROUTES=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_PEERDNS=yes
IPV6_PEERROUTES=yes
IPV6_FAILURE_FATAL=no
NAME=eno16777728
UUID=af5fbd84-b131-46cc-a92a-23c6a621025b
DEVICE=eno16777728
ONBOOT=yes
IPADDR=192.168.100.10
NETMASK=255.255.255.0
```

利用 VI 编辑器再次打开网卡文件,此网卡文件对应的 NAT 网卡,并按照下图编辑并保存。vi /etc/sysconfig/network-scripts/ifcfg-ens34

```
TYPE=Ethernet
BOOTPROTO=static
DEFROUTE=yes
PEERDNS=yes
PEERROUTES=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_PEERDNS=yes
IPV6_PEERROUTES=yes
IPV6_FAILURE_FATAL=no
NAME=eno33554952
UUID=08656c2c-bfb1-41d1-a307-dfa9e23a9221
DEVICE=eno33554952
ONBOOT=yes
IPADDR=192.168.200.10
NETMASK=255.255.255.0
```

两个网卡配置完后重启网卡

```
systemctl restart network
```

同样,编辑 compute 节点下的两个网卡对应的网卡文件。

```
vi /etc/sysconfig/network-scripts/ifcfg-eno16777728
```

```
TYPE=Ethernet
BOOTPROTO=static
DEFROUTE=yes
PEERDNS=yes
PEERROUTES=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_PEERDNS=yes
IPV6_PEERROUTES=yes
IPV6_FAILURE_FATAL=no
NAME=eno16777728
UUID=70a8038e-3d42-4c0d-a779-6df777a6e294
DEVICE=eno16777728
ONBOOT=yes
IPADDR=192.168.100.20
NETMASK=255.255.255.0
~
~
~
~
```

vi /etc/sysconfig/network-scripts/ifcfg-eno33554952

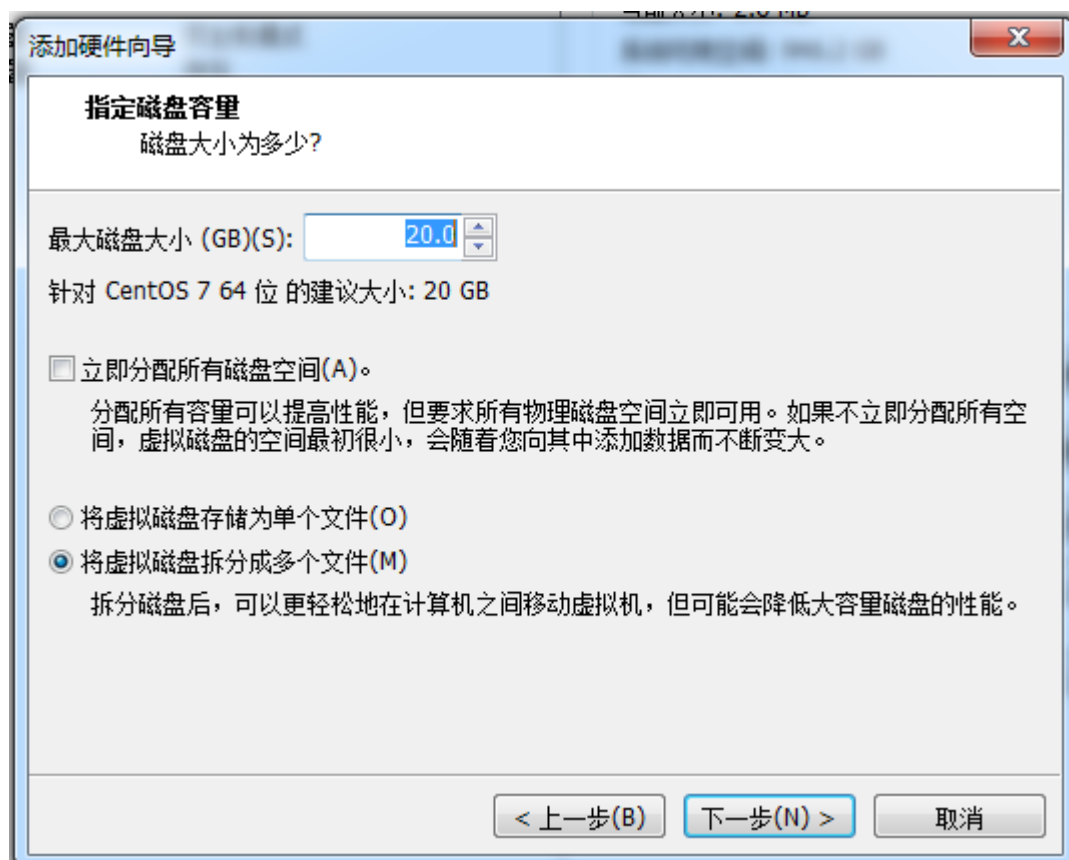
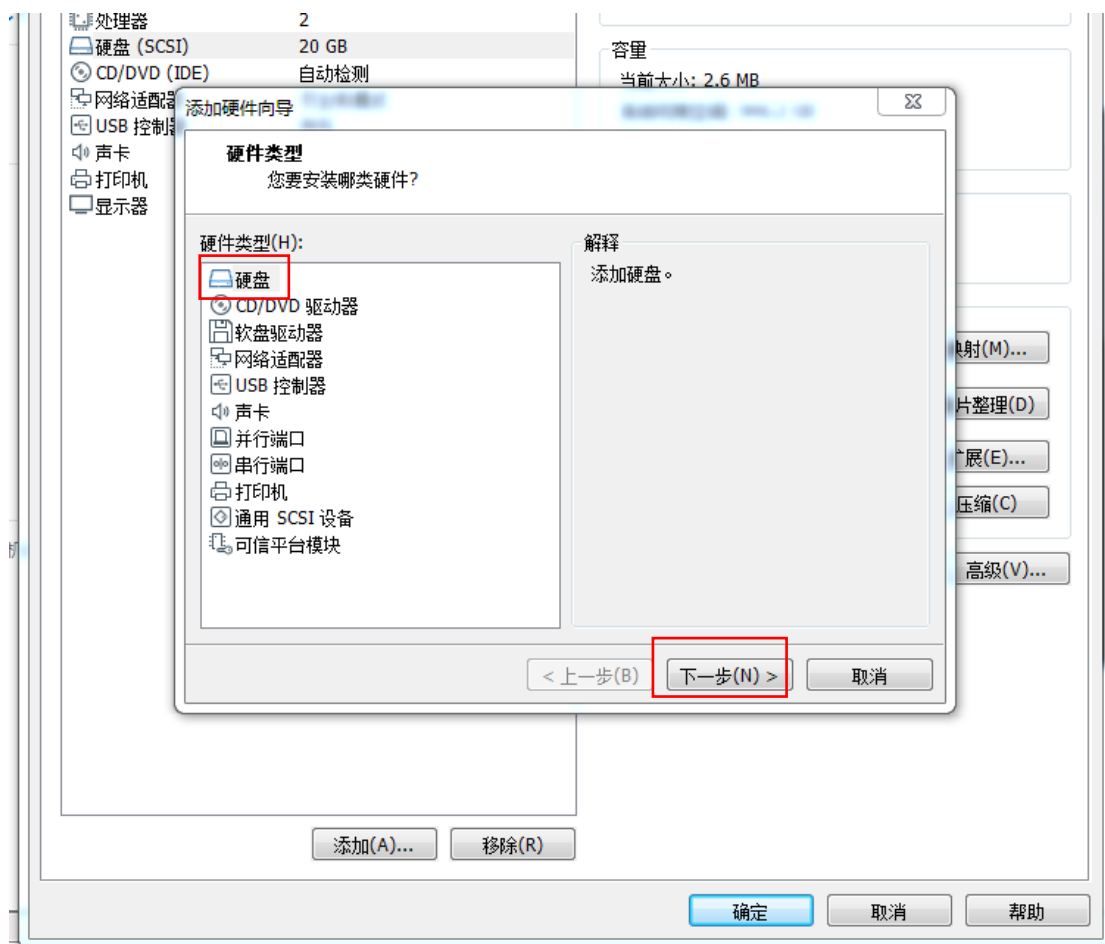
```
TYPE=Ethernet
BOOTPROTO=static
DEFROUTE=yes
PEERDNS=yes
PEERROUTES=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_PEERDNS=yes
IPV6_PEERROUTES=yes
IPV6_FAILURE_FATAL=no
NAME=eno33554952
UUID=94d076cb-c9e6-4cb5-866b-2dd034b22033
DEVICE=eno33554952
ONBOOT=yes
IPADDR=192.168.200.20
NETMASK=255.255.255.0
~
~
```

两个网卡配置完后重启网卡

systemctl restart network

4.2.2 计算节点添加硬盘，并进行分区操作

在虚拟机设置界面，给 compute 节点添加一块 20G 大小的硬盘



重启 compute 节点
在 compute 节点中：

输入 lsblk 查看磁盘

使用 fdisk 对 sdb 进行分区操作

分区命令 fdisk /dev/sdb

选择新建分区 n

选择主分区 p

连续两个回车符号（主分区号回车默认，第一个扇区回车默认）

进入分区大小设置 +10G

再次新建一个 10G 大小的主分区

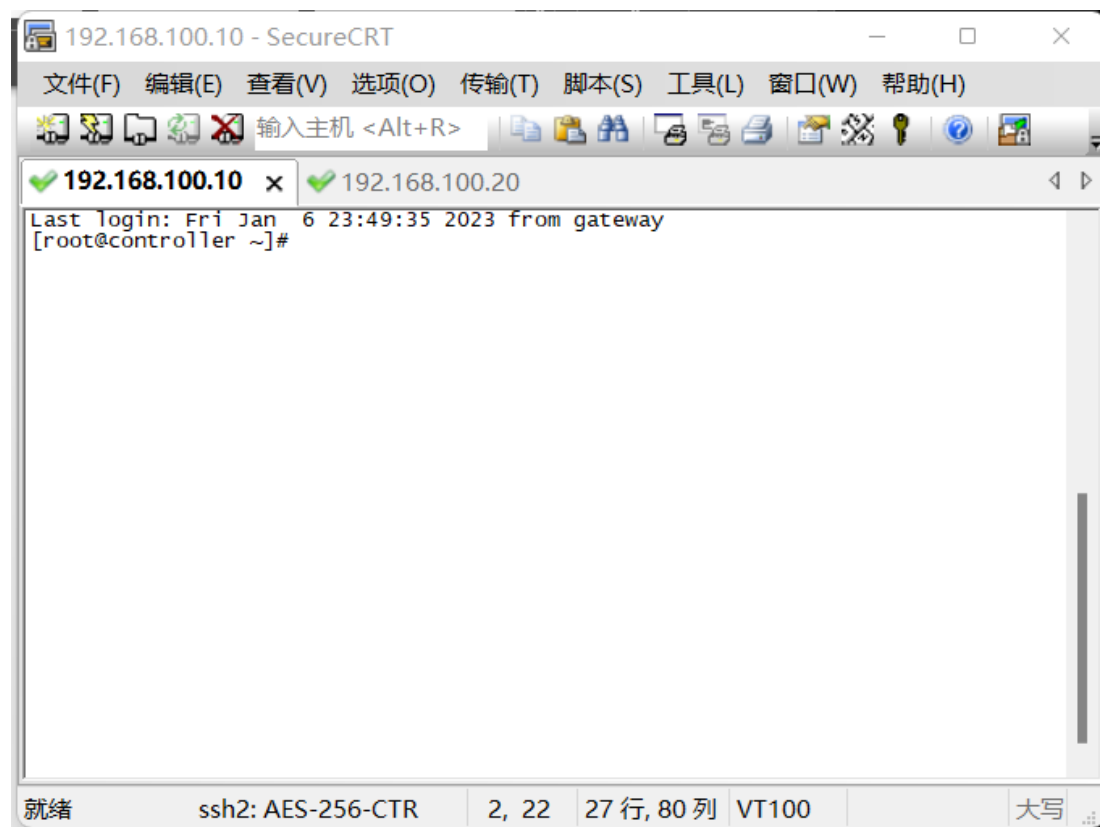
查看分区结果 P

保存分区结果 W

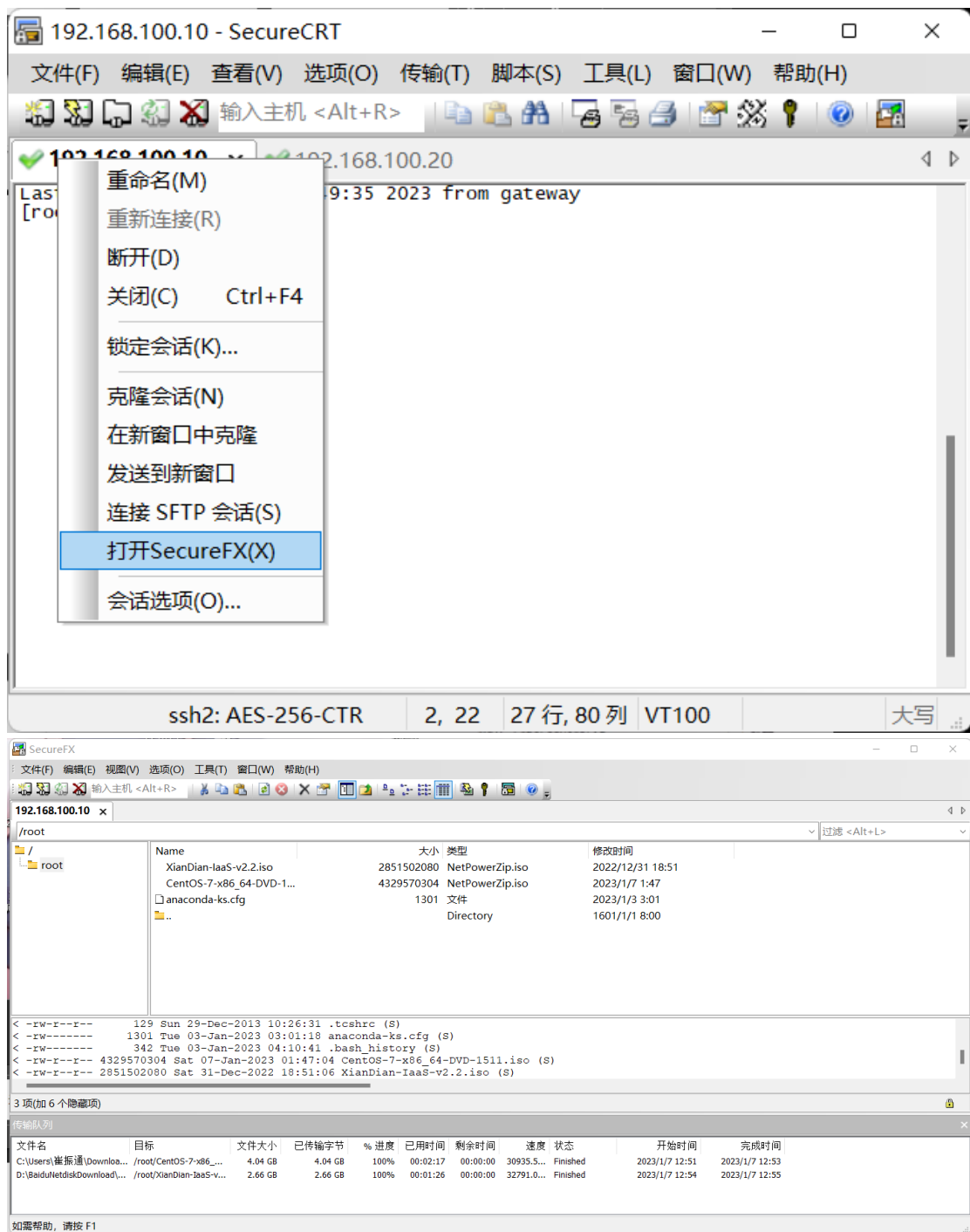
重启系统 reboot

4.2.3 上传基础镜像

使用远程连接软件 CRT，连接两主机进行远程操控



使用 CRT 对控制节点进行镜像的上传



设置挂载点，并将 centos 和 iaas 挂载到挂载点。

```
[root@controller ~]# mkdir /opt/centos7.2
[root@controller ~]# mkdir /opt/iaas
[root@controller ~]# mount /root/CentOS-7-x86_64-DVD-1511.iso /opt/centos7.2/
mount: /dev/loop0 is write-protected, mounting read-only
[root@controller ~]# mount /root/XianDian-IaaS-v2.2.iso /opt/iaas/
mount: /dev/loop1 is write-protected, mounting read-only
[root@controller ~]#
```

4.2.4 配置 yum 源文件

(1) 选择虚拟机 controller 节点

a) 利用下面命令删除原来的 yum 文件：

```
rm -rvf /etc/yum.repos.d/*
```

- b) 创建新的 yum 源文件

```
vi /etc/yum.repos.d/local.repo
```

- c) 编辑 yum 源文件，按照以下内容编辑并保存。

```
[centos]
name=centos7.2
baseurl=file:///opt/centos7.2
gpgcheck=0
enabled=1

[iaas]
name=iaas
baseurl=file:///opt/iaas/iaas-repo
gpgcheck=0
enabled=1
```

(2) 选择计算节点 compute 虚拟机

- a) 利用下面命令，删除原来的 yum 文件

```
rm -rvf /etc/yum.repos.d/*
```

- b) 创建新的 yum 源文件

```
vi /etc/yum.repos.d/local.repo
```

- c) 编辑 yum 源文件，按照以下内容编辑并保存。

```
[centos]
name=centos7.2
baseurl=ftp://192.168.100.10/centos7.2
gpgcheck=0
enabled=1

[iaas]
name=iaas
baseurl=ftp://192.168.100.10/iaas/iaas-repo
gpgcheck=0
enabled=1
```

4.2.5 安装 ftp 服务

- a) 切换虚拟机，选择 controller 控制节点

- b) 安装 ftp 服务: `yum install vsftpd -y`

- c) 打开 ftp 配置文件: `vi /etc/vsftpd/vsftpd.conf` (添加 `anon_root=/opt/`)

- d) 重启 vsftpd 服务: `systemctl restart vsftpd`

6 配置防火墙

- a) 切换虚拟机，选择 controller 控制节点

- b) 利用以下命令设置防火墙

```
setenforce 0
```

iptables -F

iptables -X

iptables -Z

systemctl stop firewalld

systemctl disable firewalld

c) 切换虚拟机，选择 computer 计算节点

d) 利用以下命令设置防火墙

```
setenforce 0
```

```
iptables -F
```

```
iptables -X
```

```
iptables -Z
```

```
systemctl stop firewalld
```

```
systemctl disable firewalld
```

4.2.6 安装 iaas-xiandian 服务

a) 切换虚拟机，选择 controller 控制节点，按照以下命令安装服务

```
yum install iaas-xiandian -y
```

b) 切换虚拟机，选择 computer 计算节点，按照以下命令安装服务

```
yum install iaas-xiandian -y
```

4.2.7 打开 openrc.sh 脚本文件，配置环境变量

a) 切换虚拟机，选择 controller 节点，按照以下命令打开 openrc.sh 脚本文件

```
vi /etc/xiandian/openrc.sh
```

b) 编辑依照图示内容修改此文件，并去除#，保存。

c) 过滤查看是否正确 `egrep -v '^#|^$' /etc/xiandian/openrc.sh`

```
[root@localhost ~]# egrep -v '^#|^$' /etc/xiandian/openrc.sh
HOST_IP=192.168.100.10
HOST_NAME=controller
HOST_IP_NODE=192.168.100.20
HOST_NAME_NODE=compute
RABBIT_USER=openstack
RABBIT_PASS=000000
DB_PASS=000000
DOMAIN_NAME=xiandian
ADMIN_PASS=000000
DEMO_PASS=000000
KEYSTONE_DBPASS=000000
GLANCE_DBPASS=000000
GLANCE_PASS=000000
NOVA_DBPASS=000000
NOVA_PASS=000000
NEUTRON_DBPASS=000000
NEUTRON_PASS=000000
METADATA_SECRET=000000
INTERFACE_NAME=eno33554952
CINDER_DBPASS=000000
CINDER_PASS=000000
BLOCK_DISK=sdb1
TROVE_DBPASS=000000
TROVE_PASS=000000
SWIFT_PASS=000000
OBJECT_DISK=sdb2
STORAGE_LOCAL_NET_IP=192.168.100.20
HEAT_DBPASS=000000
HEAT_PASS=000000
CEILOMETER_DBPASS=000000
CEILOMETER_PASS=000000
AODH_DBPASS=000000
AODH_PASS=000000
```

还有两行代码别忘了：

minvlan=

maxvlan=

d) 将控制节点的环境变量复制到计算节点

scp /etc/xiandian/openrc.sh root@192.168.100.20:/etc/xiandian/openrc.sh

4.3 安装 OpenStack 所需的库和工具

4.3.1 安装 iaas-pre-host.sh 服务

a) 切换虚拟机，选择 controller 控制节点，按照以下命令安装 host 服务

iaas-pre-host.sh

b) 切换虚拟机，选择 computer 计算节点，按照以下命令安装 host 服务

iaas-pre-host.sh

4.3.2 安装数据库服务

切换虚拟机，选择 controller 控制节点，按照以下命令安装 mysql 服务

iaas-install-mysql.sh

4.3.3 安装 Keystone 认证服务

切换虚拟机，选择 controller 控制节点，按照以下命令安装 Keystone 服务

iaas-install-keystone.sh

4.3.4 安装 Glance 镜像服务

切换虚拟机，选择 controller 控制节点，按照以下命令安装 Glance 服务
iaas-install-glance.sh

4.3.5 安装 Nova 计算服务

- a) 切换虚拟机，选择 controller 控制节点，按照以下命令安装 Nova 服务
iaas-install-nova-controller.sh
- b) 切换虚拟机，选择 computer 计算节点，按照以下命令安装 Nova 服务
iaas-install-nova-compute.sh

4.3.6 安装 Neutron 网络服务

- a) 切换虚拟机，选择 controller 控制节点，按照以下命令安装 Neutron 服务
iaas-install-neutron-controller.sh
iaas-install-neutron-controller-gre.sh
- b) 切换虚拟机，选择 computer 计算节点，按照以下命令安装 Neutron 服务
iaas-install-neutron-compute.sh
iaas-install-neutron-compute-gre.sh

4.3.7 安装 dashboard 服务

切换虚拟机，选择 controller 控制节点，按照以下命令安装 dashboard 服务
iaas-install-dashboard.sh

4.3.8 安装 Cinder 块存储服务

- a) 切换虚拟机，选择 controller 控制节点，按照以下命令安装 cinder 服务
iaas-install-cinder-controller.sh
- b) 切换虚拟机，选择 computer 计算节点，按照以下命令安装 cinder 服务
iaas-install-cinder-compute.sh

4.3.9 安装 Swift 对象存储服务

- a) 切换虚拟机，选择 controller 控制节点，按照以下命令安装 swift 服务
iaas-install-swift-controller.sh
- c) 切换虚拟机，选择 computer 计算节点，按照以下命令安装 swift 服务

iaas-install-swift-compute.sh

4.4.1 测试和验证 OpenStack 平台

打开浏览器输入 <http://192.168.100.10/dashboard>

域为 xiandian

用户名为 admin

密码为 000000



至此，成功搭建了一个完整的 OpenStack 云平台。

第五章 环境初始化配置

在完成操作系统安装与依赖软件部署后，对所有节点进行统一、标准化的环境初始化配置，确保分布式 OpenStack 平台在节点通信、时间同步、网络解析、安全策略等方面保持一致，为后续控制节点、计算节点、存储节点的部署及 OpenStack 核心组件安装提供稳定、可靠的基础运行环境。

5.1.1 关闭防火墙与 SELinux

为防止防火墙规则与 SELinux 安全策略拦截 OpenStack 各组件之间的通信，所有节点均执行关闭防火墙并永久禁用 SELinux 操作，保证服务间正常访问。

```
```bash
systemctl stop firewalld
systemctl disable firewalld
setenforce 0
sed -i 's/^SELINUX=enforcing/SELINUX=disabled/' /etc/selinux/config
```
```

5.1.2 配置主机名与节点解析

为每个节点设置唯一主机名，并在 `/etc/hosts` 文件中配置所有节点的 IP 与主机名映射关系，实现节点间无需 DNS 即可互相解析。

```
```bash
hostnamectl set-hostname controller
cat >> /etc/hosts << EOF
```

### 5.1.3 集群时间同步配置

分布式系统对时间一致性要求极高，因此所有节点安装并启动 Chrony 时间同步服务，确保全集群时间统一，避免因时间不同步导致认证、消息队列、数据库等服务异常。

```
```bash
yum install -y chrony
systemctl enable --now chronyd
chronyc sources -v
```
```



#### 5.1.4 配置静态 IP 与多平面隔离

所有节点均配置静态 IP 地址，完成\*\*管理网、数据网、存储网、外部网\*\*四网平面隔离，保证不同类型流量互不干扰，提升平台稳定性与安全性。

#### 5.1.5 检查硬件虚拟化支持

确认服务器 CPU 开启 Intel VT-x/AMD-V 硬件虚拟化功能，满足 KVM 虚拟化运行要求。

```
```bash
grep -E '(vmx|svm)' /proc/cpuinfo
```
```

### （6）SSH 免密登录配置

配置节点间 SSH 免密登录，提升后续分布式部署、服务调试与平台维护的效率。

```
```bash
ssh-keygen -t rsa
ssh-copy-id root@controller
ssh-copy-id root@compute1
ssh-copy-id root@compute2
```
```

## 第六章 分布式节点部署与配置

整体架构设计

### 6.1 节点角色划分（基于 OpenStack 三节点集群）

- **控制节点 (Controller):** OpenStack Keystone、Nova、Neutron、Glance、Placement、MySQL、RabbitMQ；部署 API 网关、负载均衡、认证鉴权、服务注册发现。
- **计算节点 (Compute):** 至少 2 台，带 GPU (A100/4090 等)，部署 vLLM/OpenStation 分布式推理服务，作为大模型 Worker 节点。
- **存储节点 (Storage):** Ceph/GlusterFS，存放模型权重、日志、缓存，支持分布式挂载。

#### 6.1.1 技术栈选型

- **OpenStack 组件:** Nova（计算调度）、Neutron（网络）、Glance（镜像）、Keystone（认证）、Cinder（块存储）。
- **大模型推理引擎:** vLLM（分布式推理、PagedAttention、张量 / 流水线并行）、OpenStation（可视化管理、自动分布式部署）。
- **分布式调度:** Ray + KubeRay（容器化、弹性扩缩容、多节点协同）。
- **API 网关:** Nginx/Traefik + FastAPI，对外提供兼容 OpenAI 的 /v1/chat/completions、/v1/completions、/v1/token API。
- **认证:** Keystone + API Key 双重认证，保障 Token API 安全。

### 6.2 OpenStack 分布式节点配置

#### 6.2.1 控制节点配置

安装依赖

```
yum install -y python3 python3-pip nginx redis
```

```
pip3 install fastapi uvicorn ray[default] kuberay-operator openstacksdk
```

# 配置 Nginx 反向代理（后续 API 网关）

```
cat > /etc/nginx/conf.d/llm-api.conf <<EOF
```

```
server {
 listen 80;
 server_name llm-api.example.com;
 location / {
 proxy_pass http://127.0.0.1:8000;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 }
}
```

```

}
EOF
systemctl enable nginx && systemctl start nginx

```

### 6.2.2 计算节点（GPU 节点）配置

```

关闭防火墙、SELinux
systemctl stop firewalld && systemctl disable firewalld
setenforce 0 && sed -i 's/SELINUX=enforcing/SELINUX=disabled/'
/etc/selinux/config

安装 NVIDIA 驱动、CUDA、Docker
yum install -y kernel-devel-$(uname -r) kernel-headers-$(uname -r)
wget
https://developer.download.nvidia.com/compute/cuda/12.2.0/local_installers/cuda-repo-rhel8-12-2-local-12.2.0-535.54.03-1.x86_64.rpm
rpm -i cuda-repo-rhel8-12-2-local-12.2.0-535.54.03-1.x86_64.rpm
yum clean all && yum -y install nvidia-driver-latest-dkms cuda
systemctl enable nvidia-persistenced && systemctl start nvidia-persistenced

```

```

安装 Docker、nvidia-docker2
yum install -y yum-utils
yum-config-manager --add-repo
https://download.docker.com/linux/centos/docker-ce.repo
yum install -y docker-ce docker-ce-cli containerd.io
distribution=$(. /etc/os-release;echo IDVERSION_ID)
curl -s -L
https://nvidia.github.io/libnvidia-container/$distribution/libnvidia-container.repo | tee /etc/yum.repos.d/nvidia-container-toolkit.repo
yum install -y nvidia-docker2
systemctl restart docker && systemctl enable docker

```

### 6.2.3 加入 OpenStack 集群

```

安装 Nova 计算服务
yum install -y openstack-nova-compute libvirt qemu-kvm

配置 nova.conf（指向控制节点）
cat > /etc/nova/nova.conf <<EOF
[DEFAULT]
enabled_apis = osapi_compute,metadata
transport_url = rabbit://openstack:RABBIT_PASS@controller

```

```
my_ip = 192.168.1.XX # 本节点 IP
use_neutron = true
firewall_driver = nova.virt.firewall.NoopFirewallDriver

[api]
auth_strategy = keystone

[keystone_authtoken]
www_authenticate_uri = http://controller:5000
auth_url = http://controller:5000
memcached_servers = controller:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = nova
password = NOVA_PASS

[vnc]
enabled = true
server_listen = 0.0.0.0
server_proxyclient_address = $my_ip
novncproxy_base_url = http://controller:6080/vnc_auto.html

[glance]
api_servers = http://controller:9292

[oslo_concurrency]
lock_path = /var/lib/nova/tmp

[placement]
region_name = RegionOne
project_domain_name = Default
project_name = service
auth_type = password
user_domain_name = Default
auth_url = http://controller:5000/v3
username = placement
password = PLACEMENT_PASS

[libvirt]
virt_type = qemu
cpu_mode = host-passthrough
EOF

启动服务
```

```
systemctl enable libvirtd openstack-nova-compute && systemctl start libvirtd
openstack-nova-compute
```

# 控制节点发现计算节点

```
source admin-openrc
```

```
su nova -s /bin/sh -c "nova-manage cell_v2 discover_hosts --verbose"
```

## 6.2.4 安装大模型推理依赖

```
pip3 install vllm ray openai transformers accelerate
```

### 5. 分布式大模型服务部署

1. 启动 Ray 集群（控制节点为 Head，计算节点为 Worker）

1.1 Head 节点（控制节点）

```
ray start --head --port=6379 --dashboard-host=0.0.0.0
```

记录 join 命令: ray start --address='controller:6379'

1.2 Worker 节点（所有 GPU 计算节点）

```
ray start --address='controller:6379' --num-gpus=1 # 按实际 GPU 数调整
```

2. 分布式 vLLM 服务部署（多节点推理）

```
```bash
```

```
from vllm import LLM, SamplingParams
```

```
import ray
```

初始化 Ray

```
ray.init(address="auto")
```

分布式 LLM（张量并行，跨节点）

```
llm = LLM(
```

```
    model="Qwen/Qwen2-7B-Instruct", # 替换为你的模型
```

```
    tensor_parallel_size=2, # 跨 2 个 GPU 节点
```

```
    distributed_executor_backend="ray",
```

```
    gpu_memory_utilization=0.9,
```

```
)
```

采样参数

```
sampling_params = SamplingParams(
```

```
    temperature=0.7,
```

```
    top_p=0.95,
```

```
    max_tokens=1024,
```

```
)
```

测试推理

```

prompts = ["你好，请介绍一下 OpenStack 集成大模型的方案"]
outputs = llm.generate(prompts, sampling_params)
for output in outputs:
    print(f"Prompt: {output.prompt}")
    print(f"Output: {output.outputs[0].text}")

```bash

```

### 6.2.5 OpenStation 可视化部署

#### 1. 控制节点安装 OpenStation:

```

docker run -d -p 8080:8080 -v /opt/models:/models
openstation/openstation:latest

```

访问 <http://controller:8080>，选择 **Distributed** 模式，选择 2 个 GPU 计算节点，上传 / 选择模型，一键部署分布式推理服务。

#### 四、Token API 服务搭建（兼容 OpenAI API）

```

```bash
from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import APIKeyHeader
from pydantic import BaseModel
from vllm import LLM, SamplingParams
import ray
import os

app = FastAPI(title="OpenStack LLM Token API", version="1.0")

# API Key 认证
api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)
VALID_API_KEYS = {"sk-xxx": "admin"} # 从 Keystone 或数据库读取

# 初始化分布式 LLM
ray.init(address="auto")
llm = LLM(
    model="Qwen/Qwen2-7B-Instruct",
    tensor_parallel_size=2,
    distributed_executor_backend="ray",
)
sampling_params = SamplingParams(temperature=0.7, max_tokens=1024)

# 请求模型
class ChatRequest(BaseModel):
    model: str

```

```

    messages: list
    temperature: float = 0.7
    max_tokens: int = 1024

# 认证依赖
def get_current_user(api_key: str = Depends(api_key_header)):
    if api_key not in VALID_API_KEYS:
        raise HTTPException(status_code=401, detail="Invalid API Key")
    return VALID_API_KEYS[api_key]

# 兼容 OpenAI 的 Chat API
@app.post("/v1/chat/completions")
async def chat_completions(request: ChatRequest, user: str =
Depends(get_current_user)):
    prompt = "\n".join([f"{msg['role']}: {msg['content']}" for msg in
request.messages])
    outputs = llm.generate(prompt, sampling_params)
    return {
        "id": "chatcmpl-xxx",
        "object": "chat.completion",
        "created": 1234567890,
        "model": request.model,
        "choices": [{"index": 0, "message": {"role": "assistant", "content":
outputs[0].outputs[0].text}}],
        "usage": {"prompt_tokens": len(prompt), "completion_tokens":
len(outputs[0].outputs[0].text), "total_tokens":
len(prompt)+len(outputs[0].outputs[0].text)}
    }

# Token 计数 API
@app.post("/v1/token/count")
async def count_tokens(text: str, user: str = Depends(get_current_user)):
    return {"token_count": len(llm.llm_engine.tokenizer.encode(text))}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```bash

```

## 6.2.6 集成 OpenStack 认证

### 1. 创建 Keystone 服务与端点

```
source admin-openrc
```

```

openstack service create --name llm --description "Large Language Model
Service" llm
openstack endpoint create --region RegionOne llm public
http://controller:8000
openstack endpoint create --region RegionOne llm internal
http://controller:8000
openstack endpoint create --region RegionOne llm admin http://controller:8000

```

## 2. 创建 LLM 角色与用户

```

openstack role create llm_user
openstack user create --domain default --password LLM_PASS llm_user1
openstack project create --domain default llm_project
openstack role add --project llm_project --user llm_user1 llm_user

```

## 3. API 服务集成 Keystone 认证（替换 API Key）

```

```bash
from keystoneauth1 import session
from keystoneauth1.identity import v3
from keystoneclient.v3 import client

def keystone_auth(token: str):
    auth = v3.Token(
        auth_url="http://controller:5000/v3",
        token=token,
        project_name="llm_project",
        project_domain_name="Default",
        user_domain_name="Default"
    )
    sess = session.Session(auth=auth)
    keystone = client.Client(session=sess)
    try:
        keystone.tokens.validate(token)
        return True
    except:
        return False
```bash

```

## 6.2.7 验证与使用

使用 API Key 调用

```

curl -X POST http://controller:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-H "X-API-Key: sk-xxx" \
-d '{

```



```
"model": "Qwen2-7B-Instruct",
"messages": [{"role": "user", "content": "介绍 OpenStack"}]
},
```

# Token 计数

```
curl -X POST http://controller:8000/v1/token/count \
-H "X-API-Key: sk-xxx" \
-d '{"text": "OpenStack 是开源云计算平台"}'
```

## 2. OpenStack Dashboard (Horizon) 集成

在 Horizon 中添加 LLM 服务面板，实现模型管理、API 调用、Token 统计可视化。

## 第七章 Keystone（身份认证服务）

Keystone 是 OpenStack 核心身份认证与授权服务，相当于云平台的“门禁中枢”，统一管理身份认证、授权、服务目录、令牌签发，所有 OpenStack 组件（Nova/Glance/Neutron 等）及外部服务（如大模型 Token API）的访问都需经其鉴权。

### 一、核心定位与四大功能

功能	作用	关键细节
身份认证 (AuthN)	验证用户 / 服务身份	支持用户名密码、API Key、Token、LDAP、SAML 联邦等；验证通过后签发临时 Token
授权(AuthZ)	控制 “能做什么”	基于 RBAC 模型，通过 <b>用户 - 角色 - 项目</b> 绑定权限（如 “创建虚拟机” “删除镜像”）
服务目录 (Catalog)	服务发现与端点管理	所有 OpenStack 服务启动时向 Keystone 注册 API 端点 (Endpoint)，用户通过 Catalog 获取服务地址
令牌管理 (Token)	维护 Token 生命周期	支持 Fernet/JWT 等格式，自动过期、刷新，保障短期凭证安全

### 7.1 核心实体

Keystone v3 是主流版本，核心实体关系清晰，支撑多租户与权限隔离：

User（用户）：访问云平台的主体（人 / 服务），如管理员、nova/glance 服务用户。

Project（项目）：旧称“租户”，资源隔离单位（如团队 / 业务线），用户需绑定项目才能操作资源。

Role（角色）：权限集合（如 admin、member、reader），通过“用户 - 角色 - 项目”绑定实现权限分配。

Domain（域）：多租户隔离边界，可管理一组用户 / 项目 / 角色（适合企业级多部门隔离）。

Endpoint（端点）：服务 API 访问地址，分 public（外部）、internal（内部）、admin（管理）三类。

Token（令牌）：临时访问凭证，含用户身份、角色、项目信息，有效期默认 24 小时（可配置）。

### 7.1.1 Nova（计算服务）

1. Nova 工作流程（创建一台 GPU 主机）
1. 用户调用 API → nova-api
2. api 鉴权（Keystone）
3. api 把请求发给消息队列
4. nova-scheduler 消费消息，选择计算节点
5. nova-conductor 写数据库
6. 目标节点 nova-compute 消费消息
7. 调用 libvirt + KVM 创建虚拟机
8. 从 Glance 拉取镜像
9. 通过 Neutron 配置网络
10. 虚拟机启动完成

### 7.1.2 Nova 关键配置文件

```
1. /etc/nova/nova.conf（核心）
[DEFAULT]
my_ip = 192.168.xxx.xxx
transport_url = rabbit://openstack:密码@controller
auth_strategy = keystone
use_neutron = true
firewall_driver = nova.virt.firewall.NoopFirewallDriver

[api]
auth_strategy = keystone

[keystone_authtoken]
auth_url = http://controller:5000/v3
memcached_servers = controller:11211
auth_type = password
project_domain_name = Default
user_domain_name = Default
project_name = service
username = nova
password = NOVA_PASS

[vnc]
enabled = true
server_listen = 0.0.0.0
novncproxy_base_url = http://controller:6080/vnc_auto.html

[glance]
api_servers = http://controller:9292
```

[database]

connection = mysql+pymysql://nova:密码@controller/nova

计算节点关键配置

[DEFAULT]

my\_ip = 192.168.100.10

transport\_url = rabbit://openstack:密码@controller

[keystone\_authtoken]

# 同控制节点

[vnc]

enabled = true

server\_listen = 0.0.0.0

server\_proxyclient\_address = \$my\_ip

novncproxy\_base\_url = http://controller:6080/vnc\_auto.html

[libvirt]

virt\_type = qemu

cpu\_mode = host-passthrough

### 3. Nova 支持 GPU

要在 OpenStack 跑大模型，必须让 Nova 管理 **NVIDIA GPU**。

1. 计算节点安装 NVIDIA 驱动 + CUDA
2. 配置 nova-compute 启用 GPU 透传 (PCI Passthrough /vGPU)

nova.conf #开启 GPU

[devices]

enabled\_vgpu\_types = nvidia-29

libvirt 配置 PCI passthrough

[nova]

passthrough\_whitelist = { "vendor\_id": "10de", "product\_id": "13c2" }

### 3. 创建 GPU 类型主机类型 (flavor)

```
openstack flavor create --vcpus 16 --ram 32768 --disk 100 --property
hw:gpu=passthrough gpu.large
```

### 4. 常用 Nova 命令

openstack server list	# 查看虚拟机
openstack server create	# 创建
openstack server delete	# 删除
openstack server start/stop/restart	# 启停
openstack server reboot	# 重启
openstack flavor list	# 云主机规格
openstack compute service list	# 计算节点状态

## 7.2 Neutron（网络服务）

### 7.2.1 Neutron 是干什么的？

Neutron = OpenStack 的虚拟网络操作系统负责：

虚拟机的 IP 分配

虚拟交换机、路由器、子网、端口

内外网互通（浮动 IP）

安全组、防火墙

多节点之间的二层 / 三层网络打通

对你的场景至关重要：分布式大模型节点（多 GPU 服务器）之间必须网络互通、低延迟、大带宽，全靠 Neutron。

### 7.2.2 核心组件

控制节点

neutron-server：API 入口，接收网络请求

neutron-dhcp-agent：给虚拟机发 IP

neutron-l3-agent：路由、SNAT、浮动 IP

neutron-metadata-agent：元数据服务

neutron-ovn-northd / OVS（现在主流 OVN）

计算节点

neutron-ovn-controller / openvswitch

neutron-openvswitch-agent

负责把虚拟机网卡接入虚拟网络

### 7.2.3 核心网络概念

#### 1. Project Network（租户网络）

项目内部私有网络，比如 192.168.1.0/24

#### 2. External Network（外部网络 / 公网）

连接物理交换机，用于浮动 IP、访问外网

#### 3. Router（路由器）

连接租户网络 ↔ 外部网络

提供 SNAT 访问互联网

#### 4. Port（端口）

虚拟机的网卡，绑定 IP、MAC、安全组

#### 5. Floating IP（浮动 IP）

外网可访问的 IP，用于大模型 API 对外提供服务

#### 6. Security Group（安全组）

放行端口：

- 8000 (大模型 Token API)
- 22 (SSH)
- 6379 (Ray 分布式调度)
- 8080 (模型管理面板)

## 7、Neutron 典型网络拓扑

外网

↑

External Network (公网网段)

↑

Router

↑

Project Network: 192.168.50.0/24 (LLM 集群专用)

↓

GPU VM1

↓

GPU VM2

↓

GPU VM3

(vLLM)

(vLLM)

(Ray Worker)

所有大模型节点在同一私有网络，低延迟互通 API 服务绑定浮动 IP 对外提供 Token API。

### 7.2.4 关键配置

neutron.conf #通用配置

[DEFAULT]

transport\_url = rabbit://openstack:pass@controller

auth\_strategy = keystone

[keystone\_auth\_token]

auth\_url = http://controller:5000/v3

username = neutron

password = NEUTRON\_PASS

project\_name = service

user\_domain\_name = Default

project\_domain\_name = Default

[database]

connection = mysql+pymysql://neutron:pass@controller/neutron

计算节点 OVS/OVN 配置

[ovs]

local\_ip = 计算节点 IP

[agent]

tunnel\_types = geneve

l2\_population = true



### 7.2.5 对你大模型部署最重要的 4 件事

#### 1. 安全组

```
openstack security group rule create --ingress --protocol tcp --dst-port 8000
llm-api-sg
openstack security group rule create --ingress --protocol tcp --dst-port 22
llm-api-sg
openstack security group rule create --ingress --protocol tcp --dst-port 6379
llm-api-sg
openstack security group rule create --ingress --protocol icmp llm-api-sg
```

## 7.3 Cinder/Swift（存储服务）

- Cinder: 块存储服务 → 对应 “云硬盘”，给虚拟机用，可挂载、可扩容、可快照。
- Swift: 对象存储服务 → 对应 “网盘 / OSS”，存文件、模型权重、日志、备份，HTTP API 访问。

你的大模型架构里两者缺一不可：

- Cinder: 给 GPU 虚拟机挂载大容量硬盘，装系统、跑服务。
- Swift: 统一存放所有大模型文件（.bin/.safetensors），多节点共享下载。

### 7.3.1 Cinder（块存储）

#### 1. 作用

- 为 Nova 虚拟机提供持久化块设备（类似阿里云云盘、AWS EBS）
- 支持：创建、挂载、卸载、扩容、快照、备份
- 后端可以是：LVM、Ceph、共享存储等

#### 2. 关键组件

- cinder-api: 接收 API 请求
- cinder-scheduler: 选择合适存储节点
- cinder-volume: 真正管理存储设备
- cinder-backup: 备份服务

#### 3. 典型使用场景（大模型必备）

系统盘不够用，挂载 500GB~2TB 数据盘放模型、环境、数据集

4. 多台 GPU 虚拟机共享存储（配合 Ceph 最佳）
5. 推理节点挂盘，避免重启丢失模型缓存

挂载命令

# 创建云硬盘

```
openstack volume create --size 2000 model-data-vol
```

# 挂载到云主机

```
openstack server add volume <server-id> <volume-id>
```

# 查看云硬盘

```
openstack volume list
```

### 7.3.2 Swift（对象存储）

## 1. 作用

- 分布式、高可用、无限扩容的**对象存储**
- 存：大模型权重、镜像文件、日志、API 访问记录、数据集
- 多节点都能通过 HTTP/HTTPS 下载，**非常适合大模型分发**

## 7. 特点

- 不支持挂载（不能像硬盘一样直接写）
- 用：容器(container) + 对象(object) 结构
- 支持签名 URL、权限控制、多副本

对你大模型的核心价值

1. **统一模型仓库**：所有版本模型存在 Swift，多 GPU 节点统一拉取
2. **不占系统盘**：避免 Cinder 盘反复扩容
3. **安全共享**：配合 Keystone 权限，只允许授权节点下载
4. 推理服务启动时：
  - Swift 下载模型 → 缓存到本地 Cinder 云盘 → 启动 vLLM

## 8. 命令

# 创建容器

```
openstack container create llm-models
```

# 上传模型文件

```
openstack object create llm-models Qwen2-7B-Instruct.tar
```

# 列出对象

```
openstack object list llm-models
```

# 下载

```
openstack object save llm-models Qwen2-7B-Instruct.tar
```

## 7.4 Horizon（可视化管理界面）

### 7.4.1 环境前提

- 已部署 OpenStack 控制节点(controller), 且 Keystone/Nova/Neutron/Cinder/Swift 正常运行
- 控制节点主机名: controller (或已配置 /etc/hosts 解析)
- 已配置好 OpenStack YUM/DNF 源 (RDO 源)
- 关闭防火墙 / SELinux (或放行 80、11211 端口)

### 7.4.2 安装依赖与 Horizon 包

```
1. 安装 Memcached (会话存储, 必须)
dnf install -y memcached python3-memcached

2. 启动并开机自启 Memcached
systemctl enable --now memcached

3. 安装 Horizon (openstack-dashboard)
dnf install -y openstack-dashboard

4. 安装 Apache (httpd) 与 WSGI 模块
dnf install -y httpd mod_wsgi
```

### 7.4.3 核心配置 (local\_settings)

编辑 Horizon 主配置文件: `vi /etc/openstack-dashboard/local_settings`

关键配置 (直接替换 / 添加)

1. 指向 OpenStack 控制节点

```
OPENSTACK_HOST = "controller"
```

2. 允许访问的主机 (生产填实际 IP/域名, 测试可用 ['\*'])

```
ALLOWED_HOSTS = ['*']
```

3. Keystone v3 认证地址 (必须)

```
OPENSTACK_KEYSTONE_URL = "http://%s:5000/v3" % OPENSTACK_HOST
```

4. 启用多域支持 (必须)

```
OPENSTACK_KEYSTONE_MULTIDOMAIN_SUPPORT = True
```

5. 默认域 (与 Keystone 一致)

```
OPENSTACK_KEYSTONE_DEFAULT_DOMAIN = "Default"
```

#### 6. 默认角色（新建用户默认角色）

```
OPENSTACK_KEYSTONE_DEFAULT_ROLE = "user"
```

#### 7. API 版本（适配当前 OpenStack）

```
OPENSTACK_API_VERSIONS = {
 "identity": 3,
 "image": 2, # Glance v2
 "volume": 2, # Cinder v2
 "compute": 2, # Nova v2
}
```

#### # 8. 会话存储：使用 Memcached（必须）

```
SESSION_ENGINE = 'django.contrib.sessions.backends.cache'
CACHES = {
 'default': {
 'BACKEND': 'django.core.cache.backends.memcached.PyMemcacheCache',
 'LOCATION': 'controller:11211',
 }
}
```

#### 9. 关闭调试（生产环境）

```
DEBUG = False
```

#### 10. 时区（中国）

```
TIME_ZONE = "Asia/Shanghai"
```

#### 11. 启用 Neutron 网络管理（必须）

```
OPENSTACK_NEUTRON_NETWORK = {
 'enable_router': True,
 'enable_quotas': True,
 'enable_ipv6': False,
 'enable_distributed_router': False,
 'enable_ha_router': False,
 'enable_lb': False,
 'enable_firewall': False,
 'enable_vpn': False,
 'enable_fip_topology_check': True,
}
```

### 7.4.4 Apache 配置 (httpd)

#### 1. 检查 WSGI 配置

```
vi /etc/httpd/conf.d/openstack-dashboard.conf
```

## 2. 数据库同步与权限

同步 Horizon 数据库 (Django)

```
su -s /bin/sh -c "openstack-dashboard-manage migrate --noinput" apache
```

修复目录权限 (关键, 否则 500 错误)

```
chown -R apache:apache /usr/share/openstack-dashboard
```

```
chown -R apache:apache /var/lib/openstack-dashboard
```

## 3. 启动服务并设置开机自启

# 重启 Apache + Memcached

```
systemctl restart httpd memcached
```

# 开机自启

```
systemctl enable httpd Memcached
```

## 结论

本次毕业设计系统地证明了开发基于分布式架构的 OpenStack 私有云计算平台在企业实际应用中的可行性。在开发过程中,巩固了 CentOS 系统运维和 OpenStack 组件部署相关技能,对使用分布式集群搭建、组件间 API 调用、大模型 API 接入等技术加深了理解。基本理解了 OpenStack 核心组件的分布式工作机制和私有云平台的设计逻辑。看到了开源项目 OpenStack 的工程性、专业性,意识到了自己与云计算领域专业开发者间的差距,确定了将来的学习方向。希望在以后的日子不断加强、改进、完善,使自己在不断学习中得到升华与成功。

## 致谢

经过了两个多月的努力，我最终完成了《基于分布式架构的 OpenStack 私有云计算平台的设计与实现》论文的写作，同时顺利完成了平台搭建、核心组件部署及大模型 API 接入的相关实践工作。从开始接到论文题目，明确研究方向与核心需求，到逐步梳理 OpenStack 分布式架构原理、设计平台整体方案，再到完成系统的搭建与调试、大模型 API 的接入与适配，最后到论文文章的撰写、修改与完善，每走一步对我来说都是新的尝试与挑战。

我首先要感谢我的论文指导老师、北京信息职业技术学院计算机与通信学院的李慧颖老师。在论文撰写和系统实现过程中，李老师始终给予我耐心的指导，针对 OpenStack 分布式部署的难点、大模型 API 接入的关键技术以及论文的结构逻辑、内容完善等方面，提出了许多有益的改善性意见，投入了超多的心血和精力，帮助我理清研究思路、解决实践中遇到的难题，让我能够顺利推进论文写作和系统开发工作。

此外，还要感谢同学和朋友们在论文编写和系统实践中提供的大力支持和帮助，在我遇到 OpenStack 组件配置故障、大模型 API 对接异常等问题时，他们积极分享经验、共同探讨解决方案，给我带来极大的启发，也让我在研究过程中不再孤单。也要感谢参考文献中的作者们，通过他们的研究文章，使我对 OpenStack 分布式架构、私有云平台设计以及大模型 API 集成等相关课题有了很好的出发点，为我的论文写作和系统实现提供了坚实的理论支撑和实践参考。

回看在北信生活的这段岁月里我经历了很多，为荣誉所努力，为集体所贡献，在专业知识的学习中不断提升自我，在实践操作中积累经验。每次想到以前的时光总是令人感动但也令人心酸，没有想到结束这么快的就到来了，曾经老师对我的帮助，同学间的互帮互助现在只剩下了无限的回忆，回忆在北信期间的每一堂课、每一次实践、每一次交流。

最后，感谢在北信的老师们和同学们，感谢他们在我的学习和生活中给予的帮助，感谢他们带来的欢声笑语，这段宝贵的时光将成为我人生中难忘的回忆，也将激励我在未来的学习和工作中继续前行，不断提升自己的专业能力，不辜负老师们的期望。



## 参考文献

- [1] Jeffery Chukwuma Obiri. Scalable Enterprise Decision-Support Platforms Using Containerized AI Workflows on Kubernetes-Openstack Infrastructure[J]. Global Academic Journal of Economics and Business, 2023, 5(6): 172-186.  
<https://www.journals.elsevier.com/journal-of-economics-and-business>
- [2] 佚名. OpenStack 平台集成指南[M]. 北京: CSDN 文库, 2026.  
<https://wenku.csdn.net/doc/7edvax9p01>
- [3] 佚名. AI 开发新姿势: OneAPI 管理 20+ 主流模型密钥[J]. CSDN 博客, 2026.  
<https://blog.csdn.net/article/details/158046592>
- [4] 佚名. 墨语灵犀部署实录: OpenStack 私有云中高可用大模型集群搭建[J]. CSDN. <https://blog.csdn.net/article/details/157861336>